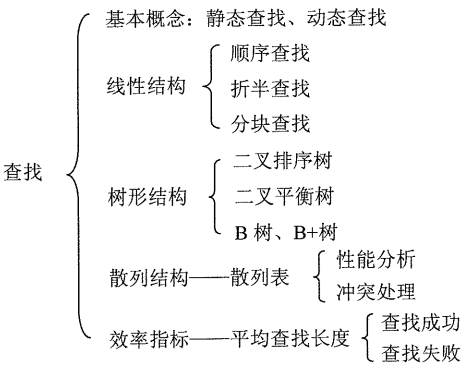
第 7 章 查找

【考纲内容】

（一）查找的基本概念
（二）顺序查找法
（三）分块查找法
（四）折半查找法
（五）B 树及其基本操作、B+树的基本概念
（六）散列表
（七）查找算法的分析及应用

【知识框架】



【复习提示】

本章是考研命题的重点。对于散列查找，应掌握散列表的构造、冲突处理方法（各种方法的处理过程）、查找成功和查找失败的平均查找长度、散列查找的特征和性能分析。对于折半查找，应掌握折半查找的过程、构造判定树、分析平均查找长度等。B 树和 B+树是本章的难点。对于 B 树，考研大纲要求掌握插入、删除和查找的操作过程；对于 B+树，仅要求了解其基本概念和性质。

7.1 查找的基本概念

1）查找。在数据集合中寻找满足某种条件的数据元素的过程称为查找。查找的结果一般分为两种：一是查找成功，即在数据集合中找到了满足条件的数据元素；二是查找失败。

2）查找表（查找结构）。用于查找的数据集合称为查找表，它由同一类型的数据元素（或记录）组成，可以是一个数组或链表等数据类型。对查找表经常进行的操作一般有 4 种：① 查询某个特定的数据元素是否在查找表中；② 检索满足条件的某个特定的数据元素

的各种属性；③ 在查找表中插入一个数据元素；④ 从查找表中删除某个数据元素。

3）静态查找表。若一个查找表的操作只涉及上述操作①和②，则无须动态地修改查找表，此类查找表称为静态查找表。与此对应，需要动态地插入或删除的查找表称为动态查找表。适合静态查找表的查找方法有顺序查找、折半查找、散列查找等；适合动态查找表的查找方法有二叉排序树的查找、散列查找等。二叉平衡树和 B 树都是二叉排序树的改进。二叉排序树与二叉平衡树已在第 4 章介绍过。

4）关键字。数据元素中唯一标识该元素的某个数据项的值，使用基于关键字的查找，查找结果应该是唯一的。例如，在由一个学生元素构成的数据集合中，学生元素中“学号”这一数据项的值唯一地标识一名学生。

5）平均查找长度。在查找过程中，一次查找的长度是指需要比较的关键字次数，而平均查找长度则是所有查找过程中进行关键字的比较次数的平均值，其数学定义为

$$\mathrm{ASL}=\sum_{i=1}^{n}P_iC_i$$

式中，n 是查找表的长度；P_i 是查找第 i 个数据元素的概率，一般认为每个数据元素的查找概率相等，即 $P_i=1/n$；C_i 是找到第 i 个数据元素所需进行的比较次数。平均查找长度是衡量查找算法效率的最主要的指标。

7.2 顺序查找和折半查找

7.2.1 顺序查找

顺序查找又称线性查找，它对顺序表和链表都是适用的。对于顺序表，可通过数组下标递增来顺序扫描每个元素；对于链表，可通过指针 next 来依次扫描每个元素。顺序查找通常分为对一般的无序线性表的顺序查找和对按关键字有序的线性表的顺序查找。下面分别进行讨论。

1．一般线性表的顺序查找

作为一种最直观的查找方法，其基本思想是从线性表的一端开始，逐个检查关键字是否满足给定的条件。若查找到某个元素的关键字满足给定条件，则查找成功，返回该元素在线性表中的位置；若已经查找到表的另一端，但还没有查找到符合给定条件的元素，则返回查找失败的信息。下面给出其算法，主要是为了说明其中引入的“哨兵”的作用。

```
typedef struct{          //查找表的数据结构
    ElemType *elem;      //元素存储空间基址，建表时按实际长度分配，0 号单元留空
    int TableLen;        //表的长度
}SSTable;
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;                                  //“哨兵”
    for(i=ST.TableLen;ST.elem[i]!=key;--i);          //从后往前找
    return  i; //若表中不存在关键字为 key 的元素，将查找到 i 为 0 时退出 for 循环
}
```

在上述算法中，将 ST.elem[0]称为“哨兵”。引入它的目的是使得 Search_Seq 内的循环不必判断数组是否会越界，因为满足 i==0 时，循环一定会跳出。需要说明的是，在程序中引入“哨兵”并不是这个算法独有的。引入“哨兵”可以避免很多不必要的判断语句，从而提高程序效率。

对于有 n 个元素的表，给定值 key 与表中第 i 个元素相等，即定位第 i 个元素时，需进行 $n-$

$i+1$ 次关键字的比较，即 $C_i=n-i+1$。查找成功时，顺序查找的平均长度为

$$\mathrm{ASL}_{\text{成功}}=\sum_{i=1}^{n}P_i(n-i+1)$$

当每个元素的查找概率相等，即 $P_i=1/n$ 时，有

$$\mathrm{ASL}_{\text{成功}}=\sum_{i=1}^{n}P_i(n-i+1)=\frac{n+1}{2}$$

查找不成功时，与表中各关键字的比较次数显然是 $n+1$ 次，从而顺序查找不成功的平均查找长度为 $\mathrm{ASL}_{\text{不成功}}=n+1$。

通常，查找表中记录的查找概率并不相等。若能预先得知每个记录的查找概率，则应先对记录的查找概率进行排序，使表中记录按查找概率由大至小重新排列。

综上所述，顺序查找的缺点是当 n 较大时，平均查找长度较大，效率低；优点是对数据元素的存储没有要求，顺序存储或链式存储皆可。对表中记录的有序性也没有要求，无论记录是否按关键字有序，均可应用。同时还需注意，对线性的链表只能进行顺序查找。

2．有序表的顺序查找

若在查找之前就已经知道表是关键字有序的，则查找失败时可以不用再比较到表的另一端就能返回查找失败的信息，从而降低顺序查找失败的平均查找长度。

假设表 L 是按关键字从小到大排列的，查找的顺序是从前往后，待查找元素的关键字为 key，当查找到第 i 个元素时，发现第 i 个元素对应的关键字小于 key，但第 $i+1$ 个元素对应的关键字大于 key，这时就可返回查找失败的信息，因为第 i 个元素之后的元素的关键字均大于 key，所以表中不存在关键字为 key 的元素。

可以用如图 7.1 所示的判定树来描述有序线性表的查找过程。树中的圆形结点表示有序线性表中存在的元素；树中的矩形结点称为失败结点（若有 n 个结点，则相应地有 $n+1$ 个查找失败结点），它描述的是那些不在表中的数据值的集合。若查找到失败结点，则说明查找不成功。

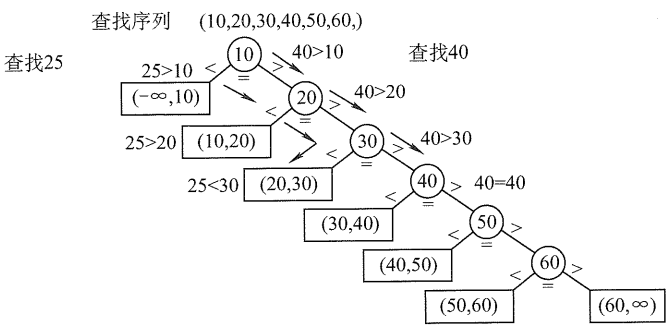


图 7.1 有序顺序表上的顺序查找判定树

在有序线性表的顺序查找中，查找成功的平均查找长度和一般线性表的顺序查找一样。查找失败时，查找指针一定走到了某个失败结点。这些失败结点是我们虚构的空结点，实际上是不存在的，所以到达失败结点时所查找的长度等于它上面的一个圆形结点的所在层数。查找不成功的平均查找长度在相等查找概率的情形下为

$$\mathrm{ASL}_{\text{不成功}}=\sum_{j=1}^{n}q_j(l_j-1)=\frac{1+2+\cdots+n+n}{n+1}=\frac{n}{2}+\frac{n}{n+1}$$

式中，q_j 是到达第 j 个失败结点的概率，在相等查找概率的情形下，它为 $1/(n+1)$；l_j 是第 j 个失

败结点所在的层数。当 $n=6$ 时，$ASL_{不成功}=6/2+6/7=3.86$，比一般的顺序查找算法好一些。

注意，有序线性表的顺序查找和后面的折半查找的思想是不一样的，且有序线性表的顺序查找中的线性表可以是链式存储结构。

7.2.2 折半查找

折半查找又称二分查找，它仅适用于有序的顺序表。

折半查找的基本思想：首先将给定值 key 与表中中间位置的元素比较，若相等，则查找成功，返回该元素的存储位置；若不等，则所需查找的元素只能在中间元素以外的前半部分或后半部分（例如，在查找表升序排列时，若给定值 key 大于中间元素，则所查找的元素只可能在后半部分）。然后在缩小的范围内继续进行同样的查找，如此重复，直到找到为止，或确定表中没有所需要查找的元素，则查找不成功，返回查找失败的信息。算法如下：

```
int Binary_Search(SeqList L,ElemType key){
    int low=0,high=L.TableLen-1,mid;
    while(low<=high){
        mid=(low+high)/2;              //取中间位置
        if(L.elem[mid]==key)
            return  mid;               //查找成功则返回所在位置
        else if(L.elem[mid]>key)
            high=mid-1;                //从前半部分继续查找
        else
            low=mid+1;                 //从后半部分继续查找
    }
    return  -1;                        //查找失败，返回-1
}
```

例如，已知 11 个元素的有序表{7, 10, 13, 16, 19, 29, 32, 33, 37, 41, 43}，要查找值为 11 和 32 的元素，指针 low 和 high 分别指向表的下界和上界，mid 则指向表的中间位置⌊(low+high)/2⌋。以下说明查找 11 的过程（查找 32 的过程请读者自行分析）：

```
          7   10   13   16   19   29   32   33   37   41   43
          ↑low               ↑mid               ↑high
```

第一次查找，将中间位置元素与 key 值比较。因为 11<29，说明待查元素若存在，则必在范围[low,mid-1]内，令指针 high 指向位置 mid-1，high=mid-1=5，重新求得 mid=(1+5)/2=3，第二次的查找范围为[1,5]。

```
          7   10   13   16   19   29   32   33   37   41   43
          ↑low    ↑mid   ↑high
```

第二次查找，同样将中间位置元素与 key 值比较。因为 11<13，说明待查元素若存在，则必在范围[low,mid-1]内，令指针 high 指向位置 mid-1，high=mid-1=2，重新求得 mid=(1+2)/2=1，第三次的查找范围为[1,2]。

```
          7   10   13   16   19   29   32   33   37   41   43
     low↑   ↑high
     mid↑
```

第三次查找，将中间位置元素与 key 值比较。因为 11>7，说明待查元素若存在，则必在范围[mid+1,high]内。令 low=mid+1=2，mid=(2+2)/2=2，第四次的查找范围为[2,2]。

```
          7   10   13   16   19   29   32   33   37   41   43
          low↑ ↑high
               ↑mid
```

第四次查找，此时子表只含有一个元素，且 10≠11，故表中不存在待查元素。

折半查找的过程可用图 7.2 所示的二叉树来描述，称为判定树。树中每个圆形结点表示一个记录，结点中的值为该记录的关键字值；树中最下面的叶结点都是方形的，它表示查找不成功的情况。从判定树可以看出，查找成功时的查找长度为从根结点到目的结点的路径上的结点数，而查找不成功时的查找长度为从根结点到对应失败结点的父结点的路径上的结点数；每个结点值均大于其左子结点值，且均小于于其右子结点值。若有序序列有 n 个元素，则对应的判定树有 n 个圆形的非叶结点和 $n+1$ 个方形的叶结点。显然，判定树是一棵平衡二叉树。

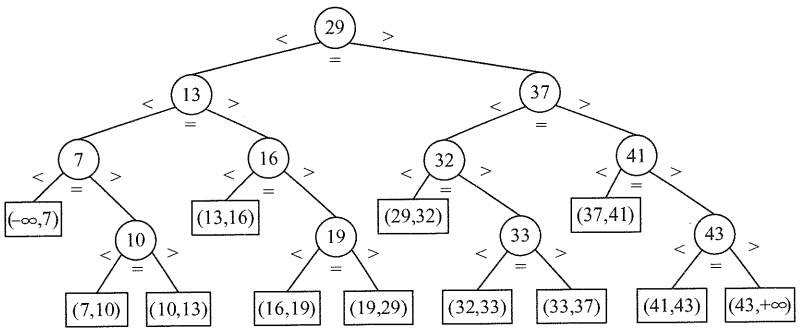


图 7.2 描述折半查找过程的判定树

由上述分析可知，用折半查找法查找到给定值的比较次数最多不会超过树的高度。在等概率查找时，查找成功的平均查找长度为

$$\mathrm{ASL}=\frac{1}{n}\sum_{i=1}^{n}l_i=\frac{1}{n}(1\times1+2\times2+\cdots+h\times2^{h-1})=\frac{n+1}{n}\log_2(n+1)-1\approx\log_2(n+1)-1$$

式中，h 是树的高度，并且元素个数为 n 时树高 $h=\lceil\log_2(n+1)\rceil$。所以，折半查找的时间复杂度为 $O(\log_2 n)$，平均情况下比顺序查找的效率高。

在图 7.2 所示的判定树中，在等概率情况下，查找成功（圆形结点）的 $\mathrm{ASL}=(1\times1+2\times2+3\times4+4\times4)/11=3$，查找不成功（方形结点）的 $\mathrm{ASL}=(3\times4+4\times8)/12=11/3$。

因为折半查找需要方便地定位查找区域，所以它要求线性表必须具有随机存取的特性。因此，该查找法仅适合于顺序存储结构，不适合于链式存储结构，且要求元素按关键字有序排列。

7.2.3 分块查找

分块查找又称索引顺序查找，它吸取了顺序查找和折半查找各自的优点，既有动态结构，又适于快速查找。

分块查找的基本思想：将查找表分为若干子块。块内的元素可以无序，但块之间是有序的，即第一个块中的最大关键字小于第二个块中的所有记录的关键字，第二个块中的最大关键字小于第三个块中的所有记录的关键字，以此类推。再建立一个索引表，索引表中的每个元素含有各块的最大关键字和各块中的第一个元素的地址，索引表按关键字有序排列。

分块查找的过程分为两步：第一步是在索引表中确定待查记录所在的块，可以顺序查找或折半查找索引表；第二步是在块内顺序查找。

例如，关键码集合为{88, 24, 72, 61, 21, 6, 32, 11, 8, 31, 22, 83, 78, 54}，按照关键码值 24, 54, 78, 88，分为 4 个块和索引表，如图 7.3 所示。

分块查找的平均查找长度为索引查找和块内查找的平均长度之和。设索引查找和块内查找的平均查找长度分别为 L_I, L_S，则分块查找的平均查找长度为

$$ASL = L_I + L_S$$

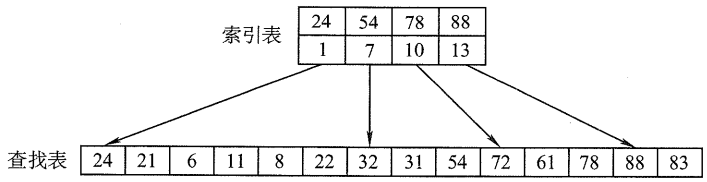


图 7.3　分块查找示意图

将长度为 n 的查找表均匀地分为 b 块，每块有 s 个记录，在等概率情况下，若在块内和索引表中均采用顺序查找，则平均查找长度为

$$ASL = L_I + L_S = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2+2s+n}{2s}$$

此时，若 $s = \sqrt{n}$，则平均查找长度取最小值 $\sqrt{n}+1$；若对索引表采用折半查找时，则平均查找长度为

$$ASL = L_I + L_S = \lceil \log_2(b+1) \rceil + \frac{s+1}{2}$$

7.2.4　本节试题精选

一、单项选择题

1. 顺序查找适合于存储结构为（ ）的线性表。

   A. 顺序存储结构或链式存储结构　　B. 散列存储结构

   C. 索引存储结构　　D. 压缩存储结构

2. 由 n 个数据元素组成的两个表：一个递增有序，一个无序。采用顺序查找算法，对有序表从头开始查找，发现当前元素已不小于待查元素时，停止查找，确定查找不成功，已知查找任一元素的概率是相同的，则在两种表中成功查找（ ）。

   A. 平均时间后者小　　B. 平均时间两者相同

   C. 平均时间前者小　　D. 无法确定

3. 对长度为 n 的有序单链表，若查找每个元素的概率相等，则顺序查找表中任一元素的查找成功的平均查找长度为（ ）。

   A. $n/2$　　B. $(n+1)/2$　　C. $(n-1)/2$　　D. $n/4$

4. 对长度为 3 的顺序表进行查找，若查找第一个元素的概率为 1/2，查找第二个元素的概率为 1/3，查找第三个元素的概率为 1/6，则查找任一元素的平均查找长度为（ ）。

   A. 5/3　　B. 2　　C. 7/3　　D. 4/3

5. 下列关于二分查找的叙述中，正确的是（ ）。

   A. 表必须有序，表可以顺序方式存储，也可以链表方式存储

   B. 表必须有序且表中数据必须是整型、实型或字符型

   C. 表必须有序，而且只能从小到大排列

   D. 表必须有序，且表只能以顺序方式存储

6. 在一个顺序存储的有序线性表上查找一个数据时，既可以采用折半查找，也可以采用顺序查找，但前者比后者的查找速度（ ）。

   A. 必然快　　B. 取决于表是递增还是递减

C. 在大部分情况下要快　　D. 必然不快

7. 折半查找过程所对应的判定树是一棵（ ）。

A. 最小生成树　　B. 平衡二叉树　　C. 完全二叉树　　D. 满二叉树

8. 【2010 统考真题】已知一个长度为 16 的顺序表 L，其元素按关键字有序排列，若采用折半查找法查找一个 L 中不存在的元素，则关键字的比较次数最多是（ ）。

A. 4　　B. 5　　C. 6　　D. 7

9. 折半查找和二叉排序树的时间性能（ ）。

A. 相同　　B. 有时不相同　　C. 完全不同　　D. 无法比较

10. 在有 11 个元素的有序表 A[1,2,…,11]中进行折半查找（⌊(low+high)/2⌋），查找元素 A[11]时，被比较的元素下标依次是（ ）。

A. 6, 8, 10, 11　　B. 6, 9, 10, 11　　C. 6, 7, 9, 11　　D. 6, 8, 9, 11

11. 已知一个有序表(13, 18, 24, 35, 47, 50, 62, 83, 90, 115, 134)，当二分查找值为 90 的元素时，查找成功的比较次数为（ ）。

A. 1　　B. 2　　C. 4　　D. 6

12. 对表长为 n 的有序表进行折半查找，其判定树的高度为（ ）。

A. $\lceil \log_2(n+1) \rceil$　　B. $\lfloor \log_2(n+1) \rfloor - 1$　　C. $\lceil \log_2 n \rceil$　　D. $\lfloor \log_2 n \rfloor - 1$

13. 已知一个长度为 16 的顺序表，其元素按关键字有序排列，若采用折半查找算法查找一个不存在的元素，则比较的次数至少是（ ），至多是（ ）。

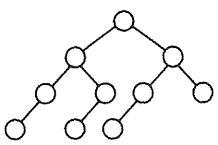
A. 4　　B. 5　　C. 6　　D. 7

14. 具有 12 个关键字的有序表中，对每个关键字的查找概率相同，折半查找算法查找成功的平均查找长度为（ ），折半查找查找失败的平均查找长度为（ ）。

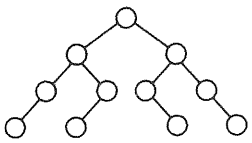
A. 37/12　　B. 35/12　　C. 39/13　　D. 49/13

15. 【2017 统考真题】下列二叉树中，可能成为折半查找判定树（不含外部结点）的是（ ）。

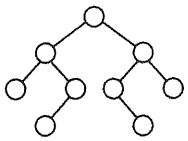
A.



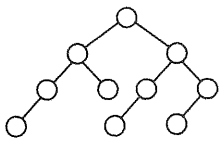
B.



C.



D.



16. 采用分块查找时，数据的组织方式为（ ）。

A. 数据分成若干块，每块内数据有序

B. 数据分成若干块，每块内数据不必有序，但块间必须有序，每块内最大（或最小）的数据组成索引块

C. 数据分成若干块，每块内数据有序，每块内最大（或最小）的数据组成索引块

D. 数据分成若干块，每块（除最后一块外）中数据个数需相同

17. 对有 2500 个记录的索引顺序表（分块表）进行查找，最理想的块长为（ ）。

A. 50　　B. 125　　C. 500　　D. $\lceil \log_2 2500 \rceil$

18. 设顺序存储的某线性表共有 123 个元素，按分块查找的要求等分为 3 块。若对索引表采用顺序查找法来确定子块，且在确定的子块中也采用顺序查找法，则在等概率情况下，分块查找成功的平均查找长度为（ ）。

A. 21　　B. 23　　C. 41　　D. 62

19. 为提高查找效率，对有 65025 个元素的有序顺序表建立索引顺序结构，在最好情况下查找到表中已有元素最多需要执行（ ）次关键字比较。

A. 10　　B. 14　　C. 16　　D. 21

20. 【2015 统考真题】下列选项中，不能构成折半查找中关键字比较序列的是（ ）。

A. 500, 200, 450, 180　　B. 500, 450, 200, 180

C. 180, 500, 200, 450　　D. 180, 200, 500, 450

21. 【2016 统考真题】在有 n（$n>1000$）个元素的升序数组 A 中查找关键字 x。查找算法的伪代码如下所示。

```
k=0;
while(k<n 且 A[k]<x) k=k+3;
if(k<n 且 A[k]==x) 查找成功;
else if(k-1<n 且 A[k-1]==x) 查找成功;
    else if(k-2<n 且 A[k-2]==x) 查找成功;
        else 查找失败;
```

本算法与折半查找算法相比，有可能具有更少比较次数的情形是（ ）。

A. 当 x 不在数组中　　B. 当 x 接近数组开头处

C. 当 x 接近数组结尾处　　D. 当 x 位于数组中间位置

二、综合应用题

1. 若对有 n 个元素的有序顺序表和无序顺序表进行顺序查找，试就下列三种情况分别讨论两者在相等查找概率时的平均查找长度是否相同。

1）查找失败。

2）查找成功，且表中只有一个关键字等于给定值 k 的元素。

3）查找成功，且表中有若干关键字等于给定值 k 的元素，要求一次查找能找出所有元素。

2. 设有序顺序表中的元素依次为 017, 094, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908。

1）试画出对其进行折半查找的判定树。

2）若查找 275 或 684 的元素，将依次与表中的哪些元素比较？

3）计算查找成功的平均查找长度和查找不成功的平均查找长度。

3. 类比二分查找算法，设计 k 分查找算法（k 为大于 2 的整数）如下：首先检查 n/k 处（n 为查找表的长度）的元素是否等于要搜索的值，然后检查 $2n/k$ 处的元素……这样，或者找到要查找的元素，或者把集合缩小到原来的 $1/k$，若未找到要查找的元素，则继续在得到的集合上进行 k 分查找；如此进行，直到找到要查找的元素或查找失败。试求查找成功和查找失败的时间复杂度。

4. 已知一个有序顺序表 A[0…8n-1]的表长为 8n，并且表中没有关键字相同的数据元素。假设按下述方法查找一个关键字值等于给定值 X 的数据元素：先在 A[7],A[15],A[23],…, A[8k-1],…,A[8n-1]中进行顺序查找，若查找成功，则算法报告成功位置并返回；若不成功，当 A[8K-1]<X<A[8×(k+1)-1]时，则可确定一个缩小的查找范围 A[8k]~ A[8× (k+1)-2]，然后可在这个范围内执行折半查找。特殊情况：若 X>A[8n-1]的关键字，则查找失败。

1）画出描述上述查找过程的判定树。

2）计算相等查找概率下查找成功的平均查找长度。

5. 【2013统考真题】设包含4个数据元素的集合$S=\{$'do', 'for', 'repeat', 'while'$\}$，各元素的查找概率依次为$p_1=0.35, p_2=0.15, p_3=0.15, p_4=0.35$。将$S$保存在一个长度为4的顺序表中，采用折半查找法，查找成功时的平均查找长度为2.2。
   1）若采用顺序存储结构保存S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？
   2）若采用链式存储结构保存S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？
6. 写出折半查找的递归算法。初始调用时，`low`为1，`high`为`ST.length`。
7. 线性表中各结点的检索概率不等时，可用如下策略提高顺序检索的效率：若找到指定的结点，则将该结点和其前驱结点（若存在）交换，使得经常被检索的结点尽量位于表的前端。试设计在顺序结构和链式结构的线性表上实现上述策略的顺序检索算法。

7.2.5 答案与解析

一、单项选择题

1．A

顺序查找是指从表的一端开始向另一端查找。它不要求查找表具有随机存取的特性，可以是顺序存储结构或链式存储结构。

2．B

对于顺序查找，不管线性表是有序的还是无序的，成功查找第一个元素的比较次数为1，成功查找第二个元素的比较次数为2，以此类推，即每个元素查找成功的比较次数只与其位置有关（与是否有序无关），因此查找成功的平均时间两者相同。

3．B

在有序单链表上做顺序查找，查找成功的平均查找长度与在无序顺序表或有序顺序表上做顺序查找的平均查找长度相同，都是$(n+1)/2$。

4．A

在长度为3的顺序表中，查找第一个元素的查找长度为1，查找第二个元素的查找长度为2，查找第三个元素的查找长度为3，故有

$$\mathrm{ASL}_{\text{成功}}=\frac{1}{2}\times 1+\frac{1}{3}\times 2+\frac{1}{6}\times 3=\frac{5}{3}$$

5．D

二分查找通过下标来定位中间位置元素，故应采用顺序存储，且二分查找能够进行的前提是查找表是有序的，但具体是从大到小还是从小到大的顺序则不做要求。

6．C

折半查找的快体现在一般情况下，在大部分情况下要快，但是对于某些特殊情况，顺序查找可能会快于折半查找。例如，查找一个含1000个元素的有序表中的第一个元素时，顺序查找的比较次数为1次，而折半查找的比较次数却将近10次。

7．B

A显然排除。对于选项C，考点精析示例中的判定树就不是完全二叉树。由选项C也可排除选项D，且满二叉树对结点数有要求。只可能选B。事实上，由折半查找的定义不难看出，每次把一个数组从中间结点分割时，总是把数组分为结点数相差最多不超过1的两个子数组，从而使得对应的判定树的两棵子树高度差的绝对值不超过1，所以应是平衡二叉树。

8．B

折半查找法在查找不成功时和给定值进行关键字的比较次数最多为树的高度，即$\lfloor \log_2 n \rfloor + 1$或$\lceil \log_2(n+1) \rceil$。在本题中，$n=16$，故比较次数最多为 5。

注意：在折半查找判定树中的方形结点是虚构的，它并不计入比较的次数中。

9．B

折半查找的性能分析可以用二叉判定树来衡量，平均查找长度和最大查找长度都是$O(\log_2 n)$；二叉排序树的查找性能与数据的输入顺序有关，最好情况下的平均查找长度与折半查找相同，但最坏情况即形成单支树时，其查找长度为$O(n)$。

10．B

依据折半查找算法的思想，第一次 mid $=\lfloor (1+11)/2 \rfloor = 6$，第二次 mid $=\lfloor [(6+1)+11]/2 \rfloor = 9$，第三次 mid $=\lfloor [(9+1)+11]/2 \rfloor = 10$，第四次 mid $= 11$。

11．B

开始时 low 指向 13，high 指向 134，mid 指向 50，比较第一次 $90 > 50$，所以将 low 指向 62，high 指向 134，mid 指向 90，第二次比较找到 90。

12．A

对 n 个结点的判定树，设结点总数 $n = 2^h - 1$，则 $h = \lceil \log_2(n+1) \rceil$。

另解：特殊值代入法。直接将 $n=1$ 和 $n=2$ 的情况代入，仅有 A 满足要求。

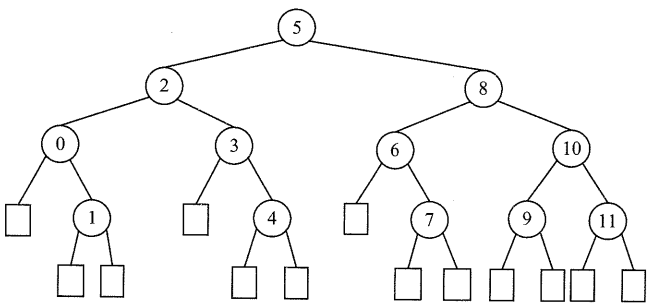
13．A、B

对于此类题，有两种做法：一种方法是，画出查找过程中构成的判定树，让最小的分支高度对应于最少的比较次数，让最大的分支高度对应于最多的比较次数，出现类似于长度为 15 的顺序表时，判定树刚好是一棵满树，此时最多比较次数与最少比较次数相等；另一种方法是，直接用公式求出最小的分支高度和最大分支高度，从前面的讲解不难看出最大分支高度为 $H = \lceil \log_2(n+1) \rceil = 5$，这对应的就是最多比较次数，然后由于判定树不是一棵满树，所以至少应该是 4（由判定树的各分支高度最多相差 1 得出）。

注意，若是求查找成功或查找失败的平均查找长度，则需要画出判定树进行求解。此外，对长度为 n 的有序表，采用折半查找时，查找成功和查找失败的最多比较次数相同，均为$\lceil \log_2(n+1) \rceil$。

14．A、D

假设有序表中元素为 A[0...11]，不难画出对它进行折半查找的判定树如下图所示，圆圈是查找成功结点，方形是虚构的查找失败结点。从而可以求出查找成功的 ASL $= (1 + 2\times2 + 3\times4 + 4\times5)/12 = 37/12$，查找失败的 ASL $= (3\times3 + 4\times10)/13$。



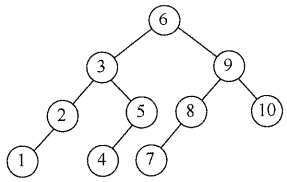
注意：对于本类题目，应先根据所给 n 的值，画出如上图的折半查找判定树。另外，查找失败结点的 ASL 不是图中的方形结点，而是方形结点上一层的圆形结点。

15．A

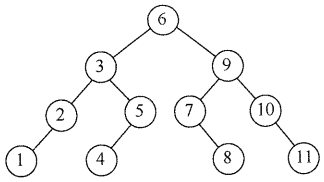
折半查找判定树实际上是一棵二叉排序树，它的中序序列是一个有序序列。可以在树结点上依次填上相应的元素，符合折半查找规则的树即为所求，如下图所示。

B 选项 4、5 相加除以 2 向上取整，7、8 相加除以 2 向下取整，矛盾。C 选项，3、4 相加除以 2 向上取整，6、7 相加除以 2 向下取整，矛盾。D 选项，1、10 相加除以 2 向下取整，6、7 相加除以 2 向上取整，矛盾。A 选项符合折半查找规则，正确。

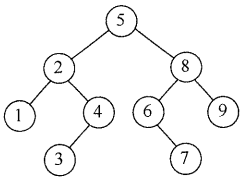
A



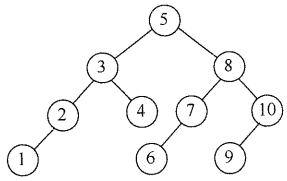
B



C



D



16．B

通常情况下，在分块查找的结构中，不要求每个索引块中的元素个数都相等。

17．A

设块长为 b，索引表包含 n/b 项，索引表的 ASL = $(n/b+1)/2$，块内的 ASL = $(b+1)/2$，总 ASL = 索引表的 ASL + 块内的 ASL = $(b+n/b+2)/2$，其中对于 $b+n/b$，由均值不等式知 $b=n/b$ 时有最小值，此时 $b=\sqrt{n}$ 。则最理想块长为 $\sqrt{2500}=50$ 。

18．B

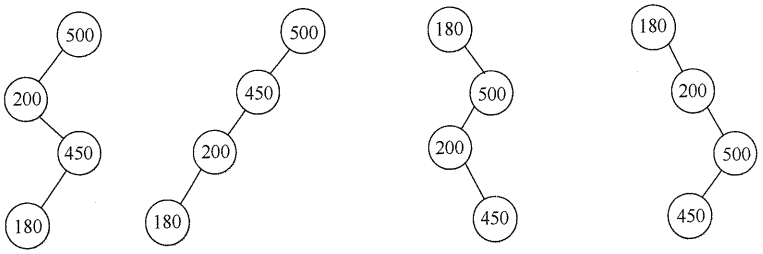
根据公式 $\mathrm{ASL}=L_1+L_S=\frac{b+1}{2}+\frac{s+1}{2}=\frac{s^2+2s+n}{2s}$，其中 $b=n/s, s=123/3, n=123$，代入不难得出 ASL 为 23。故选 B。另一方面，可根据穷举法来一步步模拟。对于 A 块中的元素，查找过程的第一步是先找到 A 块，由于是顺序查找，找到 A 块只需一步，然后在 A 块中顺序查找。因此，A 块内各元素查找长度分别为 2, 3, 4,…, 42。对于 B 块，采用类似的方法，但查找到 B 块要比查找到 A 块多一步，因此 B 块内各元素查找长度为 3, 4, 5,…, 43。同理，C 块中各个元素查找长度为 4, 5, 6,…, 44。所以平均查找长度为(2 + 3 + 4 +…+ 42 + 3 + 4 + 5 +…+ 43 + 4 + 5 + 6 +…+ 44)/123 = 23。

19．C

为使查找效率最高，每个索引块的大小应是 $\sqrt{65025}$=255 ，为每个块建立索引，则索引表中索引项的个数为 255。若对索引项和索引块内部都采用折半查找，则查找效率最高，为 $\log_2(255+1)+\log_2(255+1)=16$。

20．A

如下图所示，画出查找路径图，因为折半查找的判定树是一棵二叉排序树，因此看其是否满足二叉排序树的要求。



显然，选项 A 的查找路径不满足。

21．B

本题为送分题。该程序采用跳跃式的顺序查找法查找升序数组中的 x。显然，x 越靠前，比较次数越少。

二、综合应用题

1．解答：

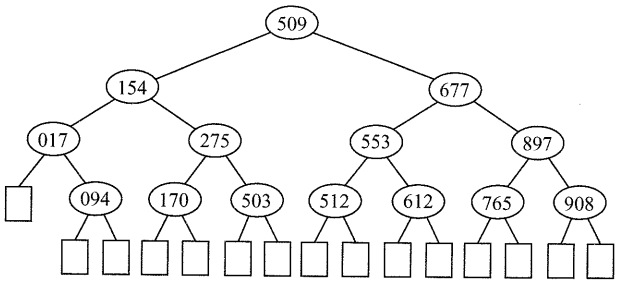
1）平均查找长度不同。因为有序顺序表查找到其关键字值比要查找值大的元素时就停止查找，并报告失败信息，不必查找到表尾；而无序顺序表必须查找到表尾才能确定查找失败。

2）平均查找长度相同。两者查找到表中元素的关键字值等于给定值时就停止查找。

3）平均查找长度不同。有序顺序表中关键字相等的元素相继排列在一起，只要查找到第一个就可以连续查找到其他关键字相同的元素。而无序顺序表必须查找全部表中的元素才能找出相同关键字的元素，因此所需的时间不同。

2．解答：

1）判定树如下图所示。



2）若查找 275，依次与表中元素 509, 154, 275 进行比较，共比较 3 次。若查找 684，依次与表中元素 509, 677, 897, 765 进行比较，共比较 4 次。

3）在查找成功时，会找到图中的某个圆形结点，其平均查找长度为

$$\mathrm{ASL}_{\text{成功}}=\frac{1}{14}\sum_{i=1}^{14}C_i=\frac{1}{14}(1+2\times2+3\times4+4\times7)=\frac{45}{14}$$

在查找失败时，会找到图中的某个方形结点，但这个结点是虚构的，最后一次的比较元素为其父结点（圆形结点），故其平均查找长度为

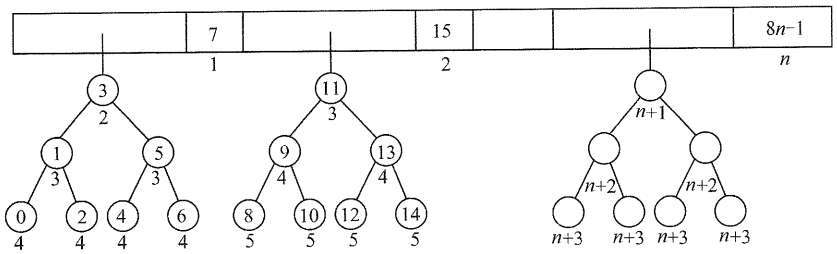
$$\mathrm{ASL}_{\text{不成功}}=\frac{1}{15}\sum_{i=0}^{14}C_i'=\frac{1}{15}(3\times1+4\times14)=\frac{59}{15}$$

3．解答：

与二分查找类似，k 分查找法可用 k 叉树来描述。k 分查找法在查找成功时进行比较的关键字的个数最多不超过树的深度，而具有 n 个结点的 k 叉树的深度为 $\lfloor\log_k n\rfloor+1$，所以 k 分查找法在查找成功时和给定值进行比较的关键字个数至多为 $\lfloor\log_k n\rfloor+1$，即时间复杂度为 $O(\log_k n)$。同理，查找不成功时，和给定值进行比较的关键字个数也至多为 $\lfloor\log_k n\rfloor+1$，故时间复杂度也为 $O(\log_k n)$。

4．解答：

1）先在 A[7],A[15],…,A[8n-1]内顺序查找，再在区间内折半查找。相应的判定树如下图所示。其中，每个关键字下的数字为其查找成功时的关键字比较次数。



2）等查找概率下，平均每个关键字查找成功的概率为 1/8n；0～7 之间的关键字，顺序比较 1 次后，进行折半查找，查找成功的平均查找长度为 2 + 3×2 + 4×4；8～15 之间的关键字，先顺序比较 2 次后，再进入折半查找；以此类推，8(n − 1)～8n − 1 之间的关键字，先顺序比较 n 次，再进入折半查找，如上图所示。故查找成功的平均查找长度为

$$\begin{aligned}ASL_{成功}&=\frac{1}{8n}\sum_{i=0}^{8n-1}C_i=\frac{1}{8n}\left(\sum_{i=1}^{n}i+\sum_{i=2}^{n+1}i+2\sum_{i=3}^{n+2}i+4\sum_{i=4}^{n+3}i\right)\\&=\frac{1}{8n}\left(\sum_{i=1}^{n}(i+(i+1)+2(i+2)+4(i+3))\right)\\&=\frac{1}{8n}\sum_{i=1}^{n}(8i+17)=\frac{1}{n}\sum_{i=1}^{n}i+\frac{17}{8}=\frac{n+1}{2}+\frac{17}{8}\end{aligned}$$

5．解答：

1）折半查找要求元素有序顺序存储，若各个元素的查找概率不同，折半查找的性能不一定优于顺序查找。采用顺序查找时，元素按其查找概率的降序排列时查找长度最小。

采用顺序存储结构，数据元素按其查找概率降序排列。采用顺序查找方法。

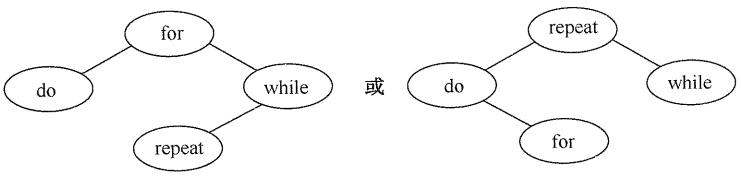
查找成功时的平均查找长度 = 0.35×1 + 0.35×2 + 0.15×3 + 0.15×4 = 2.1。

此时，显然查找长度比折半查找的更短。

2）答案一：采用链式存储结构时，只能采用顺序查找，其性能和顺序表一样，类似于上题。数据元素按其查找概率降序排列，构成单链表。采用顺序查找方法。

查找成功时的平均查找长度 = 0.35×1 + 0.35×2 + 0.15×3 + 0.15×4 = 2.1。

答案二：还可以构造成二叉排序树的形式。采用二叉链表的存储结构，构造二叉排序树，元素的存储方式见下图。采用二叉排序树的查找方法。



二叉排序树1　　二叉排序树2

查找成功时的平均查找长度 = 0.15×1 + 0.35×2 + 0.35×2 + 0.15×3 = 2.0。

6．解答：

算法的基本思想：根据查找的起始位置和终止位置，将查找序列一分为二，判断所查找的关键字在哪一部分，然后用新的序列的起始位置和终止位置递归求解。

算法代码如下：

```
typedef struct{                    //查找表的数据结构
    ElemType    *elem;             //存储空间基址，建表时按实际长度分配，0 号留空
    int         length;            //表的长度
```

```
} SSTable;
int BinSearchRec(SSTable ST,ElemType key,int low,int high){
//在有序表中递归折半查找其关键字为 key 的元素，返回其在表中序号
    if(low>high)
        return  0;
    mid=(low+high)/2;              //取中间位置
    if(key>ST.elem[mid])           //向后半部分查找
        Search(ST,key,mid+1,high);
    else if(key<ST.elem[mid])      //向前半部分查找
        Search(ST,key,low,mid-1);
    else                           //查找成功
        return  mid;
}
```

算法把规模为 n 的复杂问题经过多次递归调用转化为规模减半的子问题求解。时间复杂度为 $O(\log_2 n)$，算法中用到了一个递归工作栈，其规模与递归深度有关，也是 $O(\log_2 n)$。

7．解答：

算法的基本思想：检索时可先从表头开始向后顺序扫描，若找到指定的结点，则将该结点和其前趋结点（若存在）交换。采用顺序表存储结构的算法实现如下：

```
int SeqSrch(RcdType R[],ElemType k){
//顺序查找线性表，找到后和其前面的元素交换
    int i=0;
    while((R[i].key!=k)&&(i<n))
        i++;                       //从前向后顺序查找指定结点
    if(i<n&&i>0){                  //若找到，则交换
        temp=R[i];R[i]=R[i-1];R[i-1]=temp;
        return  --i;               //交换成功，返回交换后的位置
    }   else return  -1;           //交换失败
```

链表的实现方式请读者自行思考。注意，链表方式实现的基本思想与上述思想相似，但要注意用链表实现时，在交换两个结点之前需要保存指向前一结点的指针。

7.3 B 树和 B+树

考研大纲对 B 树和 B+树的要求各不相同，重点在于考查 B 树，不仅要求理解 B 树的基本特点，还要求掌握 B 树的建立、插入和删除操作，而对 B+树则只考查基本概念。

7.3.1 B 树及其基本操作

B 树，又称多路平衡查找树，B 树中所有结点的孩子个数的最大值称为 B 树的阶，通常用 m 表示。一棵 m 阶 B 树或为空树，或为满足如下特性的 m 叉树：

1）树中每个结点至多有 m 棵子树，即至多含有 $m-1$ 个关键字。

2）若根结点不是终端结点，则至少有两棵子树。

3）除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字。

4）所有非叶结点的结构如下：

n	P_0	K_1	P_1	K_2	P_2	$\cdots$	K_n	P_n

其中，K_i（$i=1,2,\cdots,n$）为结点的关键字，且满足 $K_1<K_2<\cdots<K_n$；P_i（$i=0,1,\cdots,n$）为

指向子树根结点的指针，且指针 P_{i-1} 所指子树中所有结点的关键字均小于 K_i，P_i 所指子树中所有结点的关键字均大于 K_i，n（$\lceil m/2\rceil - 1 \le n \le m-1$）为结点中关键字的个数。

5）所有的叶结点都出现在同一层次上，并且不带信息（可以视为外部结点或类似于折半查找判定树的查找失败结点，实际上这些结点不存在，指向这些结点的指针为空）。

B 树是所有结点的平衡因子均等于 0 的多路平衡查找树。

图 7.4 所示的 B 树中所有结点的最大孩子数 $m=5$，因此它是一棵 5 阶 B 树，在 m 阶 B 树中结点最多可以有 m 个孩子。可以借助该实例来分析上述性质：

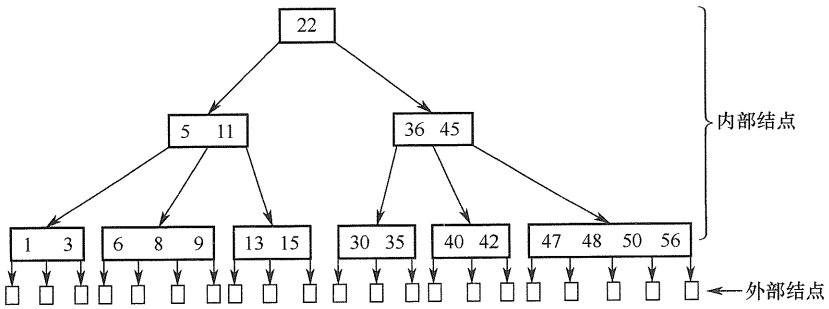


图 7.4 一棵 5 阶 B 树的实例

1）结点的孩子个数等于该结点中关键字个数加 1。

2）如果根结点没有关键字就没有子树，此时 B 树为空；如果根结点有关键字，则其子树必然大于等于两棵，因为子树个数等于关键字个数加 1。

3）除根结点外的所有非终端结点至少有$\lceil m/2\rceil = \lceil 5/2\rceil = 3$ 棵子树（即至少有$\lceil m/2\rceil - 1 = \lceil 5/2\rceil - 1 = 2$ 个关键字），至多有 5 棵子树（即至多有 4 个关键字）。

4）结点中关键字从左到右递增有序，关键字两侧均有指向子树的指针，左边指针所指子树的所有关键字均小于该关键字，右边指针所指子树的所有关键字均大于该关键字。或者看成下层结点关键字总是落在由上层结点关键字所划分的区间内，如第二层最左结点的关键字划分成了 3 个区间：$(-\infty, 5), (5, 11), (11, +\infty)$，该结点 3 个指针所指子树的关键字均落在这 3 个区间内。

5）所有叶结点均在第 4 层，代表查找失败的位置。

1．B 树的高度（磁盘存取次数）

由下一节将得知，B 树中的大部分操作所需的磁盘存取次数与 B 树的高度成正比。

下面来分析 B 树在不同情况下的高度。当然，首先应该明确 B 树的高度不包括最后的不带任何信息的叶结点所处的那一层（有些书对 B 树的高度的定义中，包含最后的那一层）。

若 $n \ge 1$，则对任意一棵包含 n 个关键字、高度为 h、阶数为 m 的 B 树：

1）因为 B 树中每个结点最多有 m 棵子树，$m-1$ 个关键字，所以在一棵高度为 h 的 m 阶 B 树中关键字的个数应满足 $n \le (m-1)(1 + m + m^2 + \cdots + m^{h-1}) = m^h - 1$，因此有

$$h \ge \log_m(n+1)$$

2）若让每个结点中的关键字个数达到最少，则容纳同样多关键字的 B 树的高度达到最大。由 B 树的定义：第一层至少有 1 个结点；第二层至少有 2 个结点；除根结点外的每个非终端结点至少有$\lceil m/2\rceil$ 棵子树，则第三层至少有 $2\lceil m/2\rceil$ 个结点……第 $h+1$ 层至少有 $2(\lceil m/2\rceil)^{h-1}$ 个结点，注意到第 $h+1$ 层是不包含任何信息的叶结点。对于关键字个数为 n 的 B 树，叶结点即查找不成功的结点为 $n+1$，由此有 $n+1 \ge 2(\lceil m/2\rceil)^{h-1}$，即 $h \le \log_{\lceil m/2\rceil}$

$((n+1)/2)+1$。

例如，假设一棵 3 阶 B 树共有 8 个关键字，则其高度范围为 $2 \leq h \leq 3.17$。

2．B 树的查找

在 B 树上进行查找与二叉查找树很相似，只是每个结点都是多个关键字的有序表，在每个结点上所做的不是两路分支决定，而是根据该结点的子树所做的多路分支决定。

B 树的查找包含两个基本操作：① 在 B 树中找结点；② 在结点内找关键字。由于 B 树常存储在磁盘上，因此前一个查找操作是在磁盘上进行的，而后一个查找操作是在内存中进行的，即在找到目标结点后，先将结点信息读入内存，然后在结点内采用顺序查找法或折半查找法。

在 B 树上查找到某个结点后，先在有序表中进行查找，若找到则查找成功，否则按照对应的指针信息到所指的子树中去查找（例如，在图 7.4 中查找关键字 42，首先从根结点开始，根结点只有一个关键字，且 $42>22$，若存在，必在关键字 22 的右边子树上，右孩子结点有两个关键字，而 $36<42<45$，则若存在，必在 36 和 45 中间的子树上，在该子结点中查到关键字 42，查找成功）。查找到叶结点时（对应指针为空指针），则说明树中没有对应的关键字，查找失败。

3．B 树的插入

与二叉查找树的插入操作相比，B 树的插入操作要复杂得多。在二叉查找树中，仅需查找到需插入的终端结点的位置。但是，在 B 树中找到插入的位置后，并不能简单地将其添加到终端结点中，因为此时可能会导致整棵树不再满足 B 树定义中的要求。将关键字 key 插入 B 树的过程如下：

1）定位。利用前述的 B 树查找算法，找出插入该关键字的最低层中的某个非叶结点（在 B 树中查找 key 时，会找到表示查找失败的叶结点，这样就确定了最底层非叶结点的插入位置。注意：插入位置一定是最低层中的某个非叶结点）。

2）插入。在 B 树中，每个非失败结点的关键字个数都在区间$[\lceil m/2\rceil-1, m-1]$内。插入后的结点关键字个数小于 m，可以直接插入；插入后检查被插入结点内关键字的个数，当插入后的结点关键字个数大于 $m-1$ 时，必须对结点进行分裂。

分裂的方法是：取一个新结点，在插入 key 后的原结点，从中间位置（$\lceil m/2\rceil$）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（$\lceil m/2\rceil$）的结点插入原结点的父结点。若此时导致其父结点的关键字个数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致 B 树高度增 1。

对于 $m=3$ 的 B 树，所有结点中最多有 $m-1=2$ 个关键字，若某结点中已有两个关键字，则结点已满，如图 7.5(a)所示。插入一个关键字 60 后，结点内的关键字个数超过了 $m-1$，如图 7.5(b)所示，此时必须进行结点分裂，分裂的结果如图 7.5(c)所示。

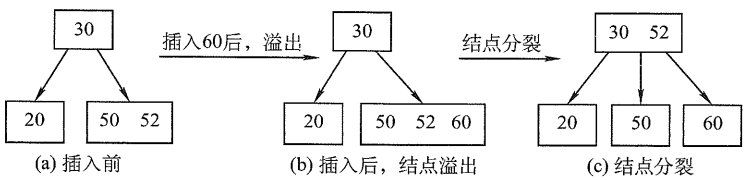


图 7.5　结点的“分裂”示意

4．B 树的删除

B 树中的删除操作与插入操作类似，但要稍微复杂一些，即要使得删除后的结点中的关键字个数$\geq\lceil m/2\rceil-1$，因此将涉及结点的“合并”问题。

当被删关键字 k 不在终端结点（最低层非叶结点）中时，可以用 k 的前驱（或后继）k'来替代 k，然后在相应的结点中删除 k'，关键字 k'必定落在某个终端结点中，则转换成了被删关键字在终端结点中的情形。在图 7.6 的 4 阶 B 树中，删除关键字 80，用其前驱 78 替代，然后在终端结点中删除 78。因此只需讨论删除终端结点中关键字的情形。

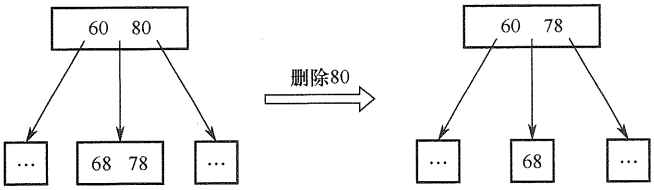
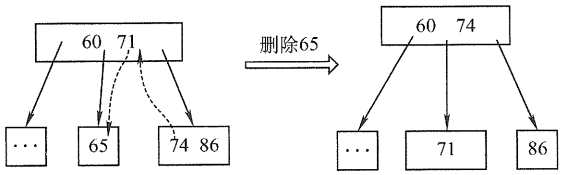


图 7.6 B 树中删除非终端结点关键字的取代

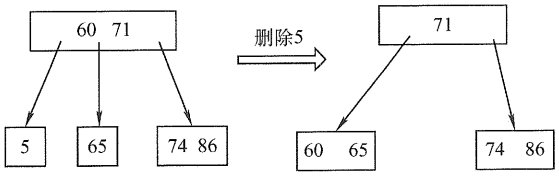
当被删关键字在终端结点（最低层非叶结点）中时，有下列三种情况：

1）直接删除关键字。若被删除关键字所在结点的关键字个数 $\geq\lceil m/2\rceil$，表明删除该关键字后仍满足 B 树的定义，则直接删去该关键字。

2）兄弟够借。若被删除关键字所在结点删除前的关键字个数 $=\lceil m/2\rceil-1$，且与此结点相邻的右（或左）兄弟结点的关键字个数 $\geq\lceil m/2\rceil$，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法），以达到新的平衡。在图 7.7(a)中删除 4 阶 B 树的关键字 65，右兄弟关键字个数 $\geq\lceil m/2\rceil=2$，将 71 取代原 65 的位置，将 74 调整到 71 的位置。



(a) 兄弟够借



(b) 兄弟不够借

图 7.7 B 树中删除终端结点关键字的示意图

3）兄弟不够借。若被删除关键字所在结点删除前的关键字个数 $=\lceil m/2\rceil-1$，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $=\lceil m/2\rceil-1$，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并。在图 7.7(b)中删除 4 阶 B 树的关键字 5，它及其右兄弟结点的关键字个数=$\lceil m/2\rceil-1=1$，故在 5 删除后将 60 合并到 65 结点中。

在合并过程中，双亲结点中的关键字个数会减 1。若其双亲结点是根结点且关键字个数减少至 0（根结点关键字个数为 1 时，有 2 棵子树），则直接将根结点删除，合并后的新结点成为根；若双亲结点不是根结点，且关键字个数减少到$\lceil m/2\rceil-2$，则又要与它自己的兄弟结点进行调整或合并操作，并重复上述步骤，直至符合 B 树的要求为止。

7.3.2 B+树的基本概念

B+树是应数据库所需而出现的一种 B 树的变形树。

一棵 m 阶的 B+树需满足下列条件：

1）每个分支结点最多有 m 棵子树（孩子结点）。

2）非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。

3）结点的子树个数与关键字个数相等。

4）所有叶结点包含全部关键字及指向相应记录的指针，叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来。

5）所有分支结点（可视为索引的索引）中仅包含它的各个子结点（即下一级的索引块）中关键字的最大值及指向其子结点的指针。

m 阶的 B+树与 m 阶的 B 树的主要差异如下：

1）在 B+树中，具有 n 个关键字的结点只含有 n 棵子树，即每个关键字对应一棵子树；而在 B 树中，具有 n 个关键字的结点含有 $n+1$ 棵子树。

2）在 B+树中，每个结点（非根内部结点）的关键字个数 n 的范围是 $\lceil m/2 \rceil \le n \le m$（根结点：$1 \le n \le m$）；在 B 树中，每个结点（非根内部结点）的关键字个数 n 的范围是 $\lceil m/2 \rceil - 1 \le n \le m-1$（根结点：$1 \le n \le m-1$）。

3）在 B+树中，叶结点包含信息，所有非叶结点仅起索引作用，非叶结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有该关键字对应记录的存储地址。

4）在 B+树中，叶结点包含了全部关键字，即在非叶结点中出现的关键字也会出现在叶结点中；而在 B 树中，叶结点（最外层内部结点）包含的关键字和其他结点包含的关键字是不重复的。

图 7.8 所示为一棵 4 阶 B+树。可以看出，分支结点的某个关键字是其子树中最大关键字的副本。通常在 B+树中有两个头指针：一个指向根结点，另一个指向关键字最小的叶结点。因此，可以对 B+树进行两种查找运算：一种是从最小关键字开始的顺序查找，另一种是从根结点开始的多路查找。

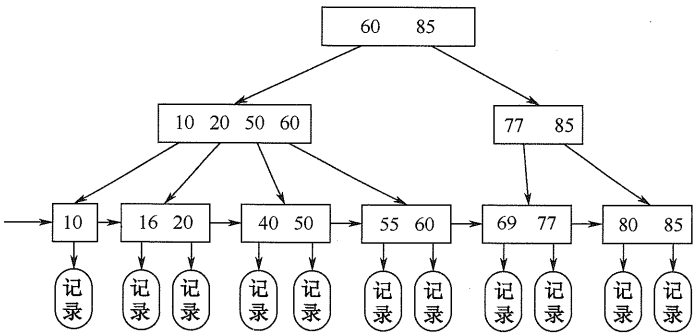


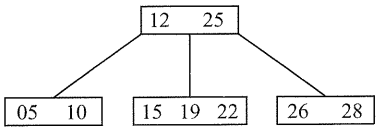
图 7.8 B+树结构示意图

B+树的查找、插入和删除操作和 B 树的基本类似。只是在查找过程中，非叶结点上的关键字值等于给定值时并不终止，而是继续向下查找，直到叶结点上的该关键字为止。所以，在 B+树中查找时，无论查找成功与否，每次查找都是一条从根结点到叶结点的路径。

7.3.3 本节试题精选

一、单项选择题

1. 下图所示是一棵（ ）。



A. 4 阶 B 树　　B. 4 阶 B+树　　C. 3 阶 B 树　　D. 3 阶 B+树

2. 下列关于 m 阶 B 树的说法中，错误的是（ ）。

A. 根结点至多有 m 棵子树

B. 所有叶结点都在同一层次上

C. 非叶结点至少有 $m/2$（m 为偶数）或$(m+1)/2$（m 为奇数）棵子树

D. 根结点中的数据是有序的

3. 以下关于 m 阶 B 树的说法中，正确的是（ ）。

I. 每个结点至少有两棵非空子树

II. 树中每个结点至多有 $m-1$ 个关键字

III. 所有叶结点在同一层

IV. 插入一个元素引起 B 树结点分裂后，树长高一层

A. I、II　　B. II、III　　C. III、IV　　D. I、II、IV

4. 【2009 统考真题】下列叙述中，不符合 m 阶 B 树定义要求的是（ ）。

A. 根结点至多有 m 棵子树　　B. 所有叶结点都在同一层上

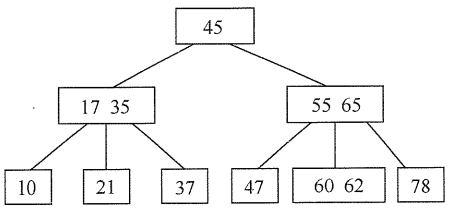
C. 各结点内关键字均升序或降序排列　　D. 叶结点之间通过指针链接

5. 在一棵 m 阶 B 树中做插入操作前，若一个结点中的关键字个数等于（ ），则必须分裂成两个结点；向一棵 m 阶的 B 树做删除操作前，若一个结点中的关键字个数等于（ ），则可能需要同它的左兄弟或右兄弟结点合并成一个结点。

A. m，$\lceil m/2 \rceil - 2$　　B. $m-1$，$\lceil m/2 \rceil - 1$

C. $m+1$，$\lceil m/2 \rceil$　　D. $m/2$，$\lceil m/2 \rceil + 1$

6. 【2012 统考真题】已知一棵 3 阶 B 树，如下图所示。删除关键字 78 得到一棵新 B 树，其最右叶结点中的关键字是（ ）。



A. 60　　B. 60, 62　　C. 62, 65　　D. 65

7. 具有 n 个关键字的 m 阶 B 树，应有（ ）个叶结点。

A. $n+1$　　B. $n-1$　　C. mn　　D. $nm/2$

8. 高度为 5 的 3 阶 B 树至少有（ ）个结点，至多有（ ）个结点。

A. 32　　B. 31　　C. 120　　D. 121

9. 【2013 统考真题】在一棵高度为 2 的 5 阶 B 树中，所含关键字的个数至少是（ ）。

A. 5　　B. 7　　C. 8　　D. 14

10. 【2014 统考真题】在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点个数最多是（ ）。

A. 5　　B. 6　　C. 10　　D. 15

11. 含有 n 个非叶结点的 m 阶 B 树中至少包含（ ）个关键字。

A. $n(m+1)$　　B. n　　C. $n(\lceil m/2\rceil-1)$　　D. $(n-1)(\lceil m/2\rceil-1)+1$

12. 已知一棵 5 阶 B 树中共有 53 个关键字，则树的最大高度为（ ），最小高度为（ ）。

A. 2　　B. 3　　C. 4　　D. 5

13. 已知一棵 3 阶 B 树中有 2047 个关键字，则此 B 树的最大高度为（ ），最小高度为（ ）。

A. 11　　B. 10　　C. 8　　D. 7

14. 下列关于 B 树和 B+树的叙述中，不正确的是（ ）。

A. B 树和 B+树都能有效地支持顺序查找

B. B 树和 B+树都能有效地支持随机查找

C. B 树和 B+树都是平衡的多叉树

D. B 树和 B+树都可以用于文件索引结构

15. 【2016 统考真题】B+树不同于 B 树的特点之一是（ ）。

A. 能支持顺序查找　　B. 结点中含有关键字

C. 根结点至少有两个分支　　D. 所有叶结点都在同一层上

16. 【2017 统考真题】下列应用中，适合使用 B+树的是（ ）。

A. 编译器中的词法分析　　B. 关系数据库系统中的索引

C. 网络中的路由表快速查找　　D. 操作系统的磁盘空闲块管理

17. 【2018 统考真题】高度为 5 的 3 阶 B 树含有的关键字个数至少是（ ）。

A. 15　　B. 31　　C. 62　　D. 242

18. 【2020 统考真题】依次将关键字 5，6，9，13，8，2，12，15 插入初始为空的 4 阶 B 树后，根结点中包含的关键字是（ ）。

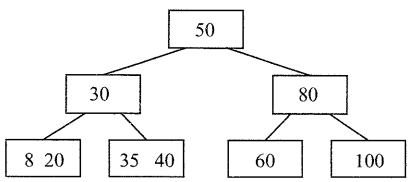
A. 8　　B. 6，9　　C. 8，13　　D. 9，12

二、综合应用题

1. 给定一组关键字{20, 30, 50, 52, 60, 68, 70}，给出创建一棵 3 阶 B 树的过程。

2. 对如下图所示的 3 阶 B 树，依次执行下列操作，画出各步操作的结果。

1）插入 90　　2）插入 25　　3）插入 45　　4）删除 60　　5）删除 80



3. 利用 B 树做文件索引时，若假设磁盘页块的大小是 4000B（实际应是 2 的次幂，此处是为了计算方便），指示磁盘地址的指针需要 5B。现有 20000000 个记录构成的文件，每个记录为 200B，其中包括关键字 5B。

试问在这个采用 B 树作索引的文件中，B 树的阶数应为多少？假定文件数据部分未按关键字有序排列，则索引部分需要占用多少磁盘页块？

7.3.4 答案与解析

一、单项选择题

1. A

关键字数目比子树数目少1，所以不是B+树，而是B树。又因为m阶B树结点关键字数最多为$m-1$，有一个结点关键字个数为3，所以不可能为3阶。

2．C

除根结点外的所有非终端结点至少有$\lceil m/2 \rceil$棵子树。对于根结点，最多有m棵子树，若其不是叶结点，则至少有2棵子树。

3．B

每个非根的内部结点必须至少有$\lceil m/2 \rceil$棵子树，而根结点至少要有两棵子树，所以I不正确。II、III显然正确。对于IV，插入一个元素引起B树结点分裂后，只要从根结点到该元素插入位置的路径上至少有一个结点未满，B树就不会长高，所以IV不正确。

4．D

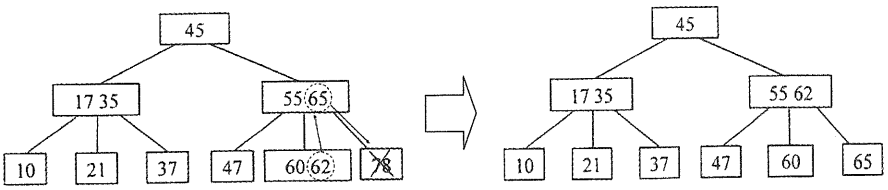
m阶B树不要求将各叶结点之间用指针链接。选项D描述的实际上是B+树。

5．B

由于B树中每个结点内的关键字个数最多为$m-1$，所以当关键字个数大于$m-1$时，就应该分裂。而每个结点内的关键字个数至少为$\lceil m/2 \rceil - 1$个，所以当关键字个数少于$\lceil m/2 \rceil - 1$时，应与其他结点合并。若将本题题干改为B+树，请读者自行思考上述问题的解答。

6．D

对于图中所示的3阶B树，被删关键字78所在的结点在删除前的关键字个数$=1=\lceil 3/2 \rceil - 1$，且其左兄弟结点的关键字个数$=2 \geq \lceil 3/2 \rceil$，属于“兄弟够借”的情况，因此要把该结点的左兄弟结点中的最大关键字上移到双亲结点中，同时把双亲结点中大于上移关键字的关键字下移到要删除关键字的结点中，这样就达到了新的平衡，如下图所示。



7．A

B树的叶结点对应查找失败的情况，对有n个关键字的查找集合进行查找，失败可能性有$n+1$种。

8．B、D

由m阶B树的性质可知，根结点至少有2棵子树；根结点外的所有非终端结点至少有$\lceil m/2 \rceil$棵子树，结点数最少时，3阶B树形状至少类似于一棵满二叉树，即高度为5的B树至少有$2^5-1=31$个结点。由于每个结点最多有m棵子树，所以当结点数最多时，3阶B树形状类似于满三叉树，结点数为$(3^5-1)/2=121$（注意，这里求的是结点数而非关键字数，若求的是关键字数，则还应把每个结点中关键字数的上下界确定出来）。

9．A

对于5阶B树，根结点只有达到5个关键字时才能产生分裂，成为高度为2的B树，因此高度为2的5阶B树所含关键字的个数最少是5。

注意：要与第1题相区别，论坛上有些同学对此题的理解存在偏差。第1题要根据图示给出阶数并指出属于哪种树。对于该题所述的5阶B树，不要误认为：“存在至少有一个含关键字的结点中的关键字达到4”才符合5阶B树的要求，因为5阶B树中的各个结点包含的关键字个数最少为2（$\lceil 5/2 \rceil - 1=2$），最多为4（$5-1=4$）。当5阶B树各个结点包含的关键字个数为2时，

也满足 5 阶 B 树的要求，存在至少有一个含关键字的结点中的关键字达到 4，同样也符合 5 阶 B 树的要求［此时若题目给定了关键字个数（如第 10 题），则可计算出该树的含关键字结点个数将达到最多；若各结点的关键字个数达到 4，则该树的总关键字个数在一定条件下可以计算出含关键字结点最少的情况。这与“当树高一定的情况下，求含关键字的最多（或最少）个数（或相反“给定关键字个数求树高”）”的思路（如 12 题）都是一样的。重中之重是对 B 树和 B+树的定义及特性一定要透彻理解。如此，相信读者方可对第 8～13 题游刃有余。对此存在疑问的读者请回看 B 树的结构及特性，不要将二者混淆］。

10．D

关键字数量不变，要求结点数量最多，即要求每个结点中含关键字的数量最少。根据 4 阶 B 树的定义，根结点最少含 1 个关键字，非根结点中最少含$\lceil 4/2 \rceil - 1 = 1$ 个关键字，所以每个结点中关键字数量最少都为 1 个，即每个结点都有 2 个分支，类似于排序二叉树，而 15 个结点正好可以构造一个 4 层的 4 阶 B 树，使得叶结点全在第四层，符合 B 树的定义，因此选 D。

11．D

除根结点外，m 阶 B 树中的每个非叶结点至少有$\lceil m/2 \rceil - 1$ 个关键字，根结点至少有一个关键字，所以总共包含的关键字最少个数 $= (n-1)(\lceil m/2 \rceil - 1) + 1$。

注意：由以上题目可知 B 树和 B+树的定义与性质尤为重要，需要熟练掌握。

12．C、B

5 阶 B 树中共有 53 个关键字，由最大高度公式 $H \leq \log_{\lceil m/2 \rceil}((n+1)/2) + 1$ 得最大高度 $H \leq \log_3[(53+1)/2] + 1 = 4$，即最大高度为 4；由最小高度公式 $h \geq \log_m(n+1)$得最小高度 $h \geq \log_5 54 = 2.5$，从而最小高度为 3。

13．A、D

利用前面的公式即最小高度 $h \geq \log_m(n+1)$和最大高度 $H \leq \log_{\lceil m/2 \rceil}[(n+1)/2] + 1$，易算出最大高度 $H \leq \log_2[(2047+1)/2] + 1 = 11$，最小高度 $h \geq \log_3 2048 = 6.9$，从而最小高度取 7（注意，有些辅导书针对本题算出的高度要比这里给出的答案多 1，因为它们在对 B 树的高度定义中，把最低层不包含任何关键字的叶结点也算进去了）。

14．A

由于 B+树的所有叶结点中包含了全部的关键字信息，且叶结点本身依关键字从小到大顺序链接，因此可以进行顺序查找，而 B 树不支持顺序查找。

15．A

由于 B+树的所有叶结点中包含了全部的关键字信息，且叶结点本身依关键字从小到大顺序链接，因此可以进行顺序查找，而 B 树不支持顺序查找（只支持多路查找）。

16．B

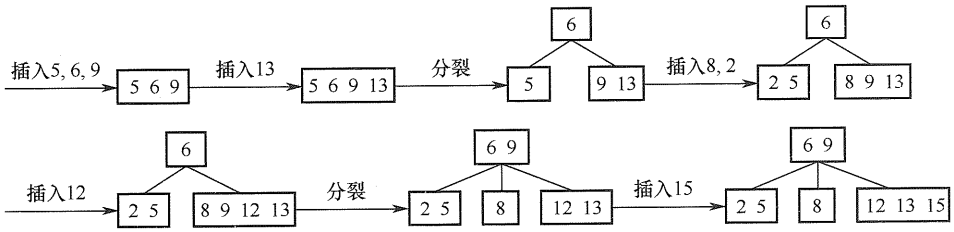
B+树是应文件系统所需而产生的 B 树的变形，前者比后者更加适用于实际应用中的操作系统的文件索引和数据库索引，因为前者的磁盘读写代价更低，查询效率更加稳定。编译器中的词法分析使用有穷自动机和语法树。网络中的路由表快速查找主要靠高速缓存、路由表压缩技术和快速查找算法。系统一般使用空闲空间链表管理磁盘空闲块。所以选项 B 正确。

17．B

m 阶 B 树的基本性质：根结点以外的非叶结点最少含有$\lceil m/2 \rceil - 1$ 个关键字，代入 $m=3$ 得到每个非叶结点中最少包含 1 个关键字，而根结点含有 1 个关键字，因此所有非叶结点都有两个孩子。此时其树形与 $h=5$ 的满二叉树相同，可求得关键字最少为 31 个。

18．B

一个4阶B树的任意非叶结点至多含有$m-1=3$个关键字，在关键字依次插入的过程中，会导致结点的不断分裂，插入过程如下所示。

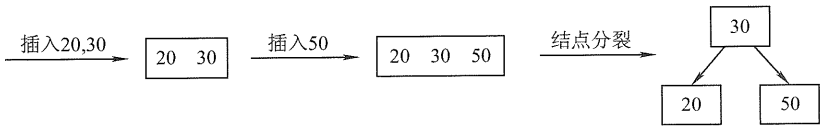


得到根结点包含的关键字为6, 9。

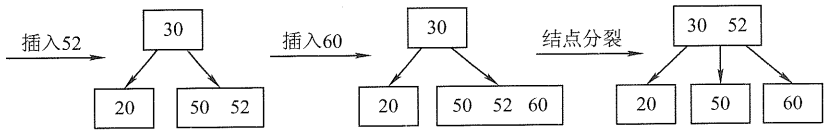
二、综合应用题

1．解答：

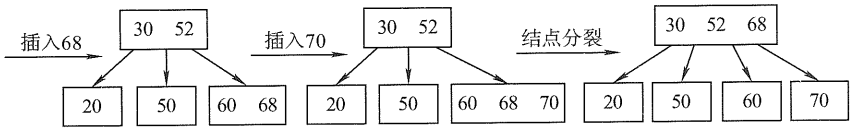
$m=3$，因此除根结点外，非叶子结点关键字个数为1～2。



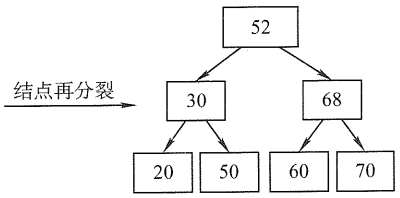
如上图所示，首先插入20, 30，结点内关键字个数不超过$\lceil m/2\rceil=2$，不会引起分裂；插入50，插入20, 30所在的结点，引起分裂，结点内第$\lceil m/2\rceil$个关键字30上升为父结点。



如上图所示，插入52，插入50所在的结点，不会引起分裂；继续插入60，插入50, 52所在的结点，引起分裂，52上升到父结点中，不会引起父结点的分裂。

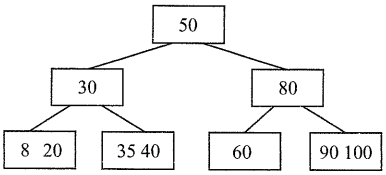


如上图所示，插入68，插入60所在的结点，不会引起分裂；继续插入70，插入60, 68所在的结点，引起分裂，68上升为新的父结点，68上升到30, 52所在的结点后，会继续引起该结点的分裂，故52上升为新的根结点。最后得到的B树如下图所示。

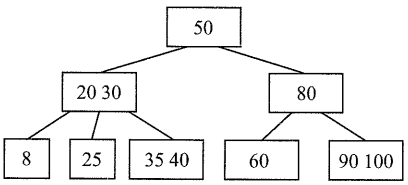


2．解答：

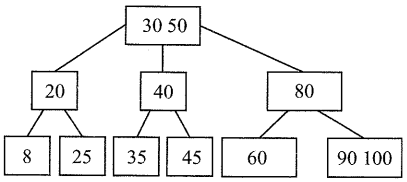
1）插入90：将90插入100所在的结点，插入90后该结点中的元素个数不超过$\lceil 3/2\rceil=2$，不会引起结点的分裂，插入后的B树如下图所示。



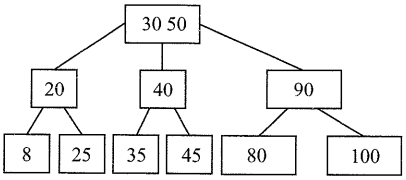
2）插入 25：将 25 插入 8, 20 所在的结点，插入后结点内的元素个数为 3，引起分裂。故将结点内的中间元素 20 上升到父结点中，此时父结点中的元素个数为 2（元素 20 和 30），不会引起继续分裂，插入 25 后的 B 树如下图所示。



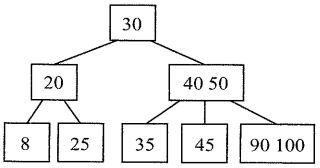
3）插入 45：将 45 插入 35, 40 所在的结点，引起分裂，中间元素 40 上升到父结点（20, 30 所在的结点）中，引起父结点分裂，中间元素 30 上升到父结点（50 所在的结点）中，两次分裂后的 B 树如下图所示。



4）删除 60：删除 60 后，其所在的结点元素为空，从而导致借用右兄弟结点的元素，调整后的 B 树如下图所示。



5）删除 80：删除 80 后，导致 80 所在结点的父结点与其右兄弟结点合并，这时父结点元素个数为 0，再次对父结点进行调整。将 50 与 40 合并成一个新结点，则 90, 100 所在结点为这个结点的子结点。从而构造的 B 树如下图所示。注意，这次调整的过程实际上包含多次调整过程，希望读者对照考点讲解中的删除过程仔细思考。



注意：B 树中结点的插入、删除操作（特别是插入、删除后的结点分裂与合并）是本节的重点，也是难点，请读者务必熟练掌握。

3．解答：

根据 B 树的概念，一个索引结点应适应操作系统一次读写的物理记录大小，其大小应取不超过但最接近一个磁盘页块的大小。假设 B 树为 m 阶，一个 B 树结点最多存放 $m-1$ 个关键字（5B）

和对应的记录地址（5B）、m 个子树指针（5B）和 1 个指示结点中的实际关键字个数的整数（2B），则有

$$(2\times(m-1)+m)\times 5+2\leq 4000$$

计算结果为 $m\leq 267$。

一个索引结点最多可以存放 $m-1=266$ 个索引项，最少可以存放 $\lceil m/2\rceil-1=133$ 个索引项。全部有 $n=20000000$ 个记录，每个记录占用空间 200B，每个页块可以存放 4000/200 = 20 个记录，则全部记录分布在 20000000/20 = 1000000 个页块中，最多需要占用 1000000/133 = 7519 个磁盘页块作为 B 树索引，最少需要占用 1000000/266 = 3760 个磁盘页块作为 B 树索引（注意 B 树与 B+树的不同，B 树所有对数据记录的索引项分布在各个层次的结点中，B+树所有对数据记录的索引项都在叶结点中）。

7.4 散列表

7.4.1 散列表的基本概念

在前面介绍的线性表和树表的查找中，记录在表中的位置与记录的关键字之间不存在确定关系，因此，在这些表中查找记录时需进行一系列的关键字比较。这类查找方法建立在“比较”的基础上，查找的效率取决于比较的次数。

散列函数：一个把查找表中的关键字映射成该关键字对应的地址的函数，记为 Hash(key) = Addr（这里的地址可以是数组下标、索引或内存地址等）。

散列函数可能会把两个或两个以上的不同关键字映射到同一地址，称这种情况为*冲突*，这些发生碰撞的不同关键字称为*同义词*。一方面，设计得好的散列函数应尽量减少这样的冲突；另一方面，由于这样的冲突总是不可避免的，所以还要设计好处理冲突的方法。

散列表：根据关键字而直接进行访问的数据结构。也就是说，散列表建立了关键字和存储地址之间的一种直接映射关系。

理想情况下，对散列表进行查找的时间复杂度为 $O(1)$，即与表中元素的个数无关。下面分别介绍常用的散列函数和处理冲突的方法。

7.4.2 散列函数的构造方法

在构造散列函数时，必须注意以下几点：

1）散列函数的定义域必须包含全部需要存储的关键字，而值域的范围则依赖于散列表的大小或地址范围。

2）散列函数计算出来的地址应该能等概率、均匀地分布在整个地址空间中，从而减少冲突的发生。

3）散列函数应尽量简单，能够在较短的时间内计算出任一关键字对应的散列地址。

下面介绍常用的散列函数。

1．直接定址法

直接取关键字的某个线性函数值为散列地址，散列函数为

$$H(\text{key})=\text{key}\ \text{或}\ H(\text{key})=a\times\text{key}+b$$

式中，a 和 b 是常数。这种方法计算最简单，且不会产生冲突。它适合关键字的分布基本连续的

情况，若关键字分布不连续，空位较多，则会造成存储空间的浪费。

2．除留余数法

这是一种最简单、最常用的方法，假定散列表表长为 m，取一个不大于 m 但最接近或等于 m 的质数 p，利用以下公式把关键字转换成散列地址。散列函数为

$$H(\text{key}) = \text{key} \% p$$

除留余数法的关键是选好 p，使得每个关键字通过该函数转换后等概率地映射到散列空间上的任一地址，从而尽可能减少冲突的可能性。

3．数字分析法

设关键字是 r 进制数（如十进制数），而 r 个数码在各位上出现的频率不一定相同，可能在某些位上分布均匀一些，每种数码出现的机会均等；而在某些位上分布不均匀，只有某几种数码经常出现，此时应选取数码分布较为均匀的若干位作为散列地址。这种方法适合于已知的关键字集合，若更换了关键字，则需要重新构造新的散列函数。

4．平方取中法

顾名思义，这种方法取关键字的平方值的中间几位作为散列地址。具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系，因此使得散列地址分布比较均匀，适用于关键字的每位取值都不够均匀或均小于散列地址所需的位数。

在不同的情况下，不同的散列函数具有不同的性能，因此不能笼统地说哪种散列函数最好。在实际选择中，采用何种构造散列函数的方法取决于关键字集合的情况，但目标是尽量降低产生冲突的可能性。

7.4.3 处理冲突的方法

应该注意到，任何设计出来的散列函数都不可能绝对地避免冲突。为此，必须考虑在发生冲突时应该如何处理，即为产生冲突的关键字寻找下一个“空”的 Hash 地址。用 H_i 表示处理冲突中第 i 次探测得到的散列地址，假设得到的另一个散列地址 H_1 仍然发生冲突，只得继续求下一个地址 H_2，以此类推，直到 H_k 不发生冲突为止，则 H_k 为关键字在表中的地址。

1．开放定址法

所谓开放定址法，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为

$$H_i = (H(\text{key}) + d_i) \% m$$

式中，$H(\text{key})$为散列函数；$i = 0, 1, 2, \cdots, k$（$k \leq m-1$）；m 表示散列表表长；d_i 为增量序列。

取定某一增量序列后，对应的处理方法就是确定的。通常有以下 4 种取法：

1）线性探测法。当 $d_i = 0, 1, 2, \cdots, m-1$ 时，称为线性探测法。这种方法的特点是：冲突发生时，顺序查看表中下一个单元（探测到表尾地址 $m-1$ 时，下一个探测地址是表首地址 0），直到找出一个空闲单元（当表未填满时一定能找到一个空闲单元）或查遍全表。

线性探测法可能使第 i 个散列地址的同义词存入第 $i+1$ 个散列地址，这样本应存入第 $i+1$ 个散列地址的元素就争夺第 $i+2$ 个散列地址的元素的地址……从而造成大量元素在相邻的散列地址上“聚集”（或堆积）起来，大大降低了查找效率。

2）平方探测法。当 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \cdots, k^2, -k^2$ 时，称为平方探测法，其中 $k \leq m/2$，散列表长度 m 必须是一个可以表示成 $4k+3$ 的素数，又称二次探测法。

平方探测法是一种处理冲突的较好方法，可以避免出现“堆积”问题，它的缺点是不能探测到散列表上的所有单元，但至少能探测到一半单元。

3）再散列法。当 $d_i = \text{Hash}_2(\text{key})$时，称为再散列法，又称双散列法。需要使用两个散列函数，当通过第一个散列函数 $H(\text{key})$得到的地址发生冲突时，则利用第二个散列函数 $\text{Hash}_2(\text{key})$计算该关键字的地址增量。它的具体散列函数形式如下：

$$H_i = (H(\text{key}) + i \times \text{Hash}_2(\text{key})) \% m$$

初始探测位置 $H_0 = H(\text{key}) \% m$。i 是冲突的次数，初始为 0。在再散列法中，最多经过 $m-1$ 次探测就会遍历表中所有位置，回到 H_0 位置。

4）伪随机序列法。当 d_i = 伪随机数序列时，称为伪随机序列法。

注意：在开放定址的情形下，不能随便物理删除表中的已有元素，因为若删除元素，则会截断其他具有相同散列地址的元素的查找地址。因此，要删除一个元素时，可给它做一个删除标记，进行逻辑删除。但这样做的副作用是：执行多次删除后，表面上看起来散列表很满，实际上有许多位置未利用，因此需要定期维护散列表，要把删除标记的元素物理删除。

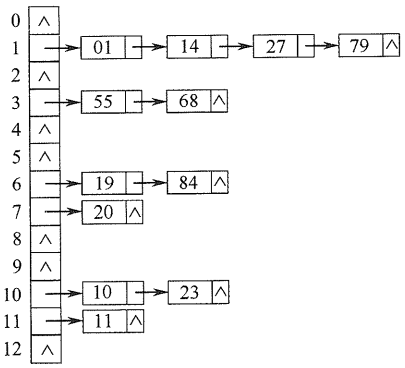


图 7.9 拉链法处理冲突的散列表

2．拉链法（链接法，chaining）

显然，对于不同的关键字可能会通过散列函数映射到同一地址，为了避免非同义词发生冲突，可以把所有的同义词存储在一个线性链表中，这个线性链表由其散列地址唯一标识。假设散列地址为 i 的同义词链表的头指针存放在散列表的第 i 个单元中，因而查找、插入和删除操作主要在同义词链中进行。拉链法适用于经常进行插入和删除的情况。

例如，关键字序列为{19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 `H(key)=key%13`，用拉链法处理冲突，建立的表如图 7.9 所示（学完下节内容后，可以尝试计算本例的平均查找长度 ASL）。

7.4.4 散列查找及性能分析

散列表的查找过程与构造散列表的过程基本一致。对于一个给定的关键字 `key`，根据散列函数可以计算出其散列地址，执行步骤如下：

初始化：`Addr=Hash(key);`

① 检测查找表中地址为 `Addr` 的位置上是否有记录，若无记录，返回**查找失败**；若有记录，比较它与 `key` 的值，若相等，则返回**查找成功**标志，否则执行步骤②。

② 用给定的处理冲突方法计算“下一个散列地址”，并把 `Addr` 置为此地址，转入步骤①。

例如，关键字序列{19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}按散列函数 `H(key)=key%13` 和线性探测处理冲突构造所得的散列表 `L` 如图 7.10 所示。

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	14	01	68	27	55	19	20	84	79	23	11	10			

图 7.10 用线性探测法得到的散列表 `L`

给定值 84 的查找过程为：首先求得散列地址 `H(84)=6`，因 `L[6]` 不空且 `L[6]≠84`，则找第一次冲突处理后的地址 H_1=`(6+1)%16=7`，而 `L[7]` 不空且 `L[7]≠84`，则找第二次冲突处理后的

地址 H_2=(6+2)%16=8，L[8]不空且 L[8]=84，查找成功，返回记录在表中的序号 8。

给定值 38 的查找过程为：先求散列地址 H(38)=12，L[12]不空且 L[12]≠38，则找下一地址 H_1=(12+1)%16=13，由于 L[13]是空记录，故表中不存在关键字为 38 的记录。

查找各关键字的比较次数如图 7.11 所示。

关键字	14	01	68	27	55	19	20	84	79	23	11	10
比较次数	1	2	1	4	3	1	1	3	9	1	1	3

图 7.11　查找各关键字的比较次数

平均查找长度 ASL 为

$$ASL=(1\times6+2+3\times3+4+9)/12=2.5$$

对同一组关键字，设定相同的散列函数，则不同的处理冲突的方法得到的散列表不同，它们的平均查找长度也不同，本例与上节采用拉链法的平均查找长度不同。

从散列表的查找过程可见：

（1）虽然散列表在关键字与记录的存储位置之间建立了直接映像，但由于“冲突”的产生，使得散列表的查找过程仍然是一个给定值和关键字进行比较的过程。因此，仍需要以平均查找长度作为衡量散列表的查找效率的度量。

（2）散列表的查找效率取决于三个因素：散列函数、处理冲突的方法和装填因子。

装填因子。散列表的装填因子一般记为α，定义为一个表的装满程度，即

$$\alpha=\frac{\text{表中记录数}n}{\text{散列表长度}m}$$

散列表的平均查找长度依赖于散列表的装填因子α，而不直接依赖于 n 或 m。直观地看，α 越大，表示装填的记录越“满”，发生冲突的可能性越大，反之发生冲突的可能性越小。

读者应能在给出散列表的长度、元素个数及散列函数和解决冲突的方法后，在求出散列表的基础上计算出查找成功时的平均查找长度和查找不成功的平均查找长度。

7.4.5　本节试题精选

一、单项选择题

1. 只能在顺序存储结构上进行的查找方法是（　）。

A. 顺序查找法　　B. 折半查找法　　C. 树型查找法　　D. 散列查找法

2. 散列查找一般适用于（　）的情况下的查找。

A. 查找表为链表

B. 查找表为有序表

C. 关键字集合比地址集合大得多

D. 关键字集合与地址集合之间存在对应关系

3. 下列关于散列表的说法中，正确的是（　）。

I. 若散列表的填装因子 $\alpha<1$，则可避免碰撞的产生

II. 散列查找中不需要任何关键字的比较

III. 散列表在查找成功时平均查找长度与表长有关

IV. 若在散列表中删除一个元素，不能简单地将该元素删除

A. I 和 IV　　B. II 和 III　　C. III　　D. IV

4. 在开放定址法中散列到同一个地址而引起的“堆积”问题是由于（ ）引起的。
   A. 同义词之间发生冲突
   B. 非同义词之间发生冲突
   C. 同义词之间或非同义词之间发生冲突
   D. 散列表“溢出”
5. 下列关于散列冲突处理方法的说法中，正确的有（ ）。
   I. 采用再散列法处理冲突时不易产生聚集
   II. 采用线性探测法处理冲突时，所有同义词在散列表中一定相邻
   III. 采用链地址法处理冲突时，若限定在链首插入，则插入任一个元素的时间是相同的
   IV. 采用链地址法处理冲突易引起聚集现象
   A. I和III　　B. I、II和III　　C. III和IV　　D. I和IV
6. 设有一个含有200个表项的散列表，用线性探测法解决冲突，按关键字查询时找到一个表项的平均探测次数不超过1.5，则散列表项应能够容纳（ ）个表项（设查找成功的平均查找长度为ASL = [1 + 1/(1 − α)]/2，其中α为装填因子）。
   A. 400　　B. 526　　C. 624　　D. 676
7. 假定有K个关键字互为同义词，若用线性探测法把这K个关键字填入散列表，至少要进行（ ）次探测。
   A. $K-1$　　B. K　　C. $K+1$　　D. $K(K+1)/2$
8. 对包含n个元素的散列表进行查找，平均查找长度（ ）。
   A. 为$O(\log_2 n)$　　B. 为$O(1)$　　C. 不直接依赖于n　　D. 直接依赖于表长m
9. 采用开放定址法解决冲突的散列查找中，发生聚集的原因主要是（ ）。
   A. 数据元素过多　　B. 负载因子过大
   C. 散列函数选择不当　　D. 解决冲突的方法选择不当
10. 【2011统考真题】为提高散列表的查找效率，可以采取的正确措施是（ ）。
    I. 增大装填（载）因子
    II. 设计冲突（碰撞）少的散列函数
    III. 处理冲突（碰撞）时避免产生聚集（堆积）现象
    A. 仅I　　B. 仅II　　C. 仅I、II　　D. 仅II、III
11. 【2014统考真题】用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚集）现象，下列选项中，会受堆积现象直接影响的是（ ）。
    A. 存储效率　　B. 散列函数
    C. 装填（装载）因子　　D. 平均查找长度
12. 一组记录的关键字为{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，用链地址法构造散列表，散列函数为H(key)=key MOD 13，散列地址为1的链中有（ ）个记录。
    A. 1　　B. 2　　C. 3　　D. 4
13. 在采用链地址法处理冲突所构成的散列表上查找某一关键字，则在查找成功的情况下，所探测的这些位置上的键值（ ）；若采用线性探测法，则（ ）。
    A. 一定都是同义词　　B. 不一定都是同义词
    C. 都相同　　D. 一定都不是同义词
14. 若采用链地址法构造散列表，散列函数为H(key)=key MOD 17，则需（①）个链表。这些链的链首指针构成一个指针数组，数组的下标范围为（②）。

①A. 17　　B. 13　　C. 16　　D. 任意

②A. 0～17　　B. 1～17　　C. 0～16　　D. 1～16

15. 设散列表长 $m=14$，散列函数为 H(key)=key%11，表中仅有 4 个结点 H(15)=4，H(38)=5，H(61)=6，H(84)=7，若采用线性探测法处理冲突，则关键字为 49 的结点地址是（ ）。

A. 8　　B. 3　　C. 5　　D. 9

16. 将 10 个元素散列到 100000 个单元的散列表中，则（ ）产生冲突。

A. 一定会　　B. 一定不会　　C. 仍可能会　　D. 不确定

17. 【2018 统考真题】现有长度为 7、初始为空的散列表 HT，散列函数 H(k)=k %7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入 HT 后，查找成功的平均查找长度是（ ）。

A. 1.5　　B. 1.6　　C. 2　　D. 3

18. 【2019 统考真题】现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key)=key%7，采用线性探查（线性探测再散列）法解决冲突。将关键字序列 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是（ ）。

A. 4　　B. 5.25　　C. 6　　D. 6.29

二、综合应用题

1. 若要在散列表中删除一个记录，应如何操作？为什么？
2. 假定把关键字 key 散列到有 n 个表项（从 0 到 $n-1$ 编址）的散列表中。对于下面的每个函数 H(key)（key 为整数），这些函数能够当作散列函数吗？若能，它是一个好的散列函数吗？说明理由。设函数 random(n)返回一个 0 到 $n-1$ 之间的随机整数（包括 0 与 $n-1$ 在内）。

   1）H(key)=key/n。

   2）H(key)=1。

   3）H(key)=(key+random(n))%n。

   4）H(key)=key%p(n)；其中 p(n)是不大于 n 的最大素数。
3. 使用散列函数 H(key)=key%11，把一个整数值转换成散列表下标，现要把数据{1, 13, 12, 34, 38, 33, 27, 22}依次插入散列表。

   1）使用线性探测法来构造散列表。

   2）使用链地址法构造散列表。

   试针对这两种情况，分别确定查找成功所需的平均查找长度，及查找不成功所需的平均查找长度。
4. 已知一组关键字为{26, 36, 41, 38, 44, 15, 68, 12, 6, 51, 25}，用链地址法解决冲突，假设装填因子 $\alpha=0.75$，散列函数的形式为 H(key)=key%P，回答以下问题：

   1）构造出散列函数。

   2）分别计算出等概率情况下查找成功和查找失败的平均查找长度（查找失败的计算中只将与关键字的比较次数计算在内即可）。
5. 【2010 统考真题】将关键字序列(7, 8, 30, 11, 18, 9, 14)散列存储到散列表中。散列表的存储空间是一个下标从 0 开始的一维数组，散列函数为 H(key)=(key×3)MOD 7，处理冲突采用线性探测再散列法，要求装填（载）因子为 0.7。

1）请画出所构造的散列表。

2）分别计算等概率情况下，查找成功和查找不成功的平均查找长度。

6. 设散列表为 HT[0...12]，即表的大小为 $m = 13$。现采用双散列法解决冲突，散列函数和再散列函数分别为：

H_0(key)=key%13　　注：%是求余数运算（=MOD）

$H_i=(H_{i-1}+REV(key+1)\%11+1)\%13$；　$i=1,2,3,\cdots,m-1$

其中，函数 REV(x)表示颠倒十进制数 x 的各位，如 REV(37)=73，REV(7)=7 等。若插入的关键码序列为(2, 8, 31, 20, 19, 18, 53, 27)，请回答：

1）画出插入这 8 个关键码后的散列表。

2）计算查找成功的平均查找长度 ASL。

7.4.6 答案与解析

一、单项选择题

1．B

顺序查找可以是顺序存储或链式存储；折半查找只能是顺序存储且要求关键字有序；树形查找法要求采用树的存储结构，既可以采用顺序存储也可以采用链式存储；散列查找中的链地址法解决冲突时，采用的是顺序存储与链式存储相结合的方式。

2．D

关键字集合与地址集合之间存在对应关系时，通过散列函数表示这种关系。这样，查找以计算散列函数而非比较的方式进行查找。

3．D

冲突（碰撞）是不可避免的，与装填因子无关，因此需要设计处理冲突的方法，I 错误。散列查找的思想是计算出散列地址来进行查找，然后比较关键字以确定是否查找成功，II 错误。散列查找成功的平均查找长度与装填因子有关，与表长无关，III 错误。在开放定址的情形下，不能随便删除散列表中的某个元素，否则可能会导致搜索路径被中断（因此通常的做法是在要删除的地方做删除标记，而不是直接删除），IV 正确。

4．C

在开放定址法中散列到同一个地址而产生的“堆积”问题，是同义词冲突的探查序列和非同义词之间不同的探查序列交织在一起，导致关键字查询需要经过较长的探测距离，降低了散列的效率。因此要选择好的处理冲突的方法来避免“堆积”。

5．A

利用再散列法处理冲突时，按一定的距离，跳跃式地寻找“下一个”空闲位置，减少了发生聚集的可能，I 正确。散列地址 i 的关键字，和为解决冲突形成的某次探测地址为 i 的关键字，都争夺地址 $i, i+1,\cdots$，因此不一定相邻，II 错误。III 正确。同义词冲突不等于聚集，链地址法处理冲突时将同义词放在同一个链表中，不会引起聚集现象，IV 错误。

6．A

若有 200 个表项要放入散列表，采用线性探测法解决冲突，限定查找成功的平均查找长度不超过 1.5，则

$$ASL_{成功}=\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)\le 1.5 \Rightarrow \alpha=\frac{200}{m}\le\frac{1}{2}\Rightarrow m\ge 400$$

7．D

K 个关键字在依次填入的过程中，只有第一个不会发生冲突，故探测次数为$(1+2+3+\cdots+K)=K(K+1)/2$，即选 D。

8．C

在散列表中，平均查找长度与装填因子α直接相关，表的查找效率不直接依赖于表中已有表项个数 n 或表长 m。若散列表中存放的记录全部是某个地址的同义词，则平均查找长度为 $O(n)$而非 $O(1)$。

9．D

聚集是因选取不当的处理冲突的方法，而导致不同关键字的元素对同一散列地址进行争夺的现象。用线性再探测法时，容易引发聚集现象。

10．D

散列表的查找效率取决于：散列函数、处理冲突的方法和装填因子。显然，冲突的产生概率与装填因子（即表中记录数与表长之比）的大小成正比，I 与题意相反。II 显然正确。采用合适的冲突处理方法可避免聚集现象，也将提高查找效率，III 正确。例如，用链地址法处理冲突时不存在聚集现象，用线性探测法处理冲突时易引起聚集现象。

11．D

产生堆积现象，即产生了冲突，它对存储效率、散列函数和装填因子均不会有影响，而平均查找长度会因为堆积现象而增大，选 D。

12．D

由散列函数计算可知，14, 1, 27, 79 散列后的地址都是 1，所以有 4 个记录。

13．A，B

因为在链地址法中，映射到同一地址的关键字都会链到与此地址相对应的链表上，所以探测过程一定是在此链表上进行的，从而这些位置上的关键字均为同义词；但在线性探测法中出现两个同义关键字时，会把该关键字对应地址的下一个地址也占用掉，两个地址分别记为 `Addr`、`Addr+1`，查找一个满足 `H(key)=Addr+1` 的关键字 `key` 时，显然首次探测到的不是 `key` 的同义词。

14．A，C

H 的取值有 17 种可能，对应到不同的链表中，所以链表的个数应为 17。由于 `H(key)` 的取值范围是 0～16，所以数组下标为 0～16。

15．A

线性探测法的公式为 $H_i=(H(\text{key})+d_i)\%m$，其中 $d_i=1, 2, 3, \cdots, m-1$。$H(49)=49\%11=5$，发生冲突；$H_1=(H(49)+1)\%14=6$，发生冲突；$H_2=(H(49)+2)\%14=7$，发生冲突；$H_3=(H(49)+3)\%14=8$，无冲突。选 A。

16．C

由于散列函数的选取，仍然有可能产生地址冲突，冲突不能绝对地避免。

17．C

根据题意，得到的 HT 如下：

0	1	2	3	4	5	6
	22	43	15			

$ASL_{成功}=(1+2+3)/3=2$。

18．C

采用线性探查法计算每个关键字的存放情况如下表所示。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	98	22	30	87	11	40	6	20			

由于$H(\text{key})=0\sim6$，查找失败时可能对应的地址有7个，对于计算出地址为0的关键字key0，只有比较完0～8号地址后才能确定该关键字不在表中，比较次数为9；对于计算出地址为1的关键字key1，只有比较完1～8号地址后才能确定该关键字不在表中，比较次数为8；以此类推。需要特别注意的是，散列函数不可能计算出地址7，因此有

$$\text{ASL}_{\text{失败}}=(9+8+7+6+5+4+3)/7=6$$

二、综合应用题

1．解答：

在散列表中删除一个记录，在拉链法情况下可以物理地删除。但在开放定址法情况下，不能物理地删除，只能做删除标记。该地址可能是该记录的同义词查找路径上的地址，物理地删除就中断了查找路径，因为查找时碰到空地址就认为是查找失败。

2．解答：

1）不能作为散列函数，因为key/n可能大于n，这样就无法找到适合的位置。

2）能够作为散列函数，但不是一个好的散列函数，因为所有关键字都映射到同一位置，造成大量的冲突机会。

3）不能当作散列函数，因为该函数的返回值不确定，这样无法进行正常的查找。

4）能够作为散列函数，是一个好的散列函数。

3．分析：

由散列函数可知散列地址的范围为0～10。

采用线性探测法构造散列表时，首先应计算出关键字对应的散列地址，然后检查散列表中对应的地址是否已经有元素。若没有元素，则直接将该关键字放入散列表对应的地址中；若有元素，则采用线性探测的方法查找下一个地址，从而决定该关键字的存放位置。

采用链地址法构造散列表时，在直接计算出关键字对应的散列地址后，将关键字结点插入此散列地址所在的链表。

解答：

1）线性探测法。

$H(1)=1$，无冲突，地址1存放关键字1。$H(13)=2$，无冲突，地址2存放关键字13。$H(12)=1$，发生冲突，根据线性探测法：$H_1=2$，发生冲突，继续探测$H_2=3$，无冲突，于是12存放在地址为3的表项中。$H(34)=1$，发生冲突，根据线性探测法：$H_1=2$，发生冲突，$H_2=3$，发生冲突，$H_3=4$，没有冲突，于是34存放在地址为4的表项中。

同理，可以计算其他的数据存放情况，最后结果如下表所示。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	33	1	13	12	34	38	27	22			
冲突次数	0	0	0	2	3	0	1	7			

下面计算平均查找长度：

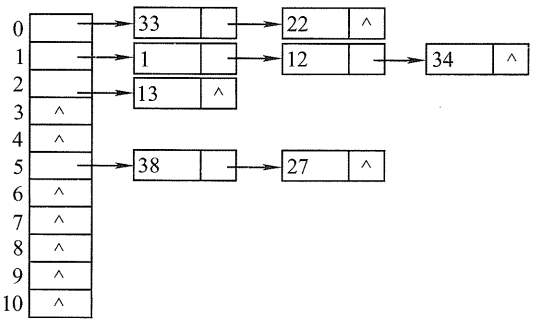
查找成功时，显然查找每个元素的概率都是 1/8。对于 33，由于冲突次数为 0，所以仅需 1 次比较便可查找成功；对于 22，由于计算出的地址为 0，但需要 8 次比较才能查找成功，所以 22 的查找长度为 8；其他元素的分析类似。因此有

$$ASL_{成功} = (1+1+1+3+4+1+2+8)/8 = 21/8$$

查找失败时，由于 $H(key) = 0 \sim 10$，因此对每个位置查找的概率都是 1/11，对于计算出的地址为 0 的关键字 key0，只有探测完 0～8 号地址后才能确定该元素不在表中，比较次数为 9；对于计算出的地址为 1 的关键字 key1，只有探测完 1～8 号地址后，才能确定该元素不在表中，比较次数为 8，以此类推。而对于计算出的地址为 8, 9, 10 的关键字，这些单元中没有存放元素，所以只需比较 1 次便可确定查找失败，因此有

$$ASL_{失败} = (9+8+7+6+5+4+3+2+1+1+1)/11 = 47/11$$

2）链地址法构造的表如下：



在链地址表中查找成功时，查找关键字为 33 的记录需进行 1 次比较，查找关键字为 22 的记录需进行 2 次比较，以此类推。因此有

$$ASL_{成功} = (1\times4 + 2\times3 + 3)/8 = 13/8$$

查找失败时，对于地址 0，比较 3 次后确定元素不在表中（空指针算 1 次），所以其查找长度为 3；对于地址 1，其查找长度为 4；对于地址 2，查找长度为 2；以此类推。因此有

$$ASL_{失败} = (3+4+2+1+1+3+1+1+1+1+1)/11 = 19/11$$

值得注意的是，求查找失败的平均查找长度有两种观点：其一，认为比较到空结点才算失败，所以比较次数等于冲突次数加 1；其二，认为只有与关键字的比较才算比较次数。

4．分析：

由装填因子的计算公式 $\alpha = n/N$（n 为关键字个数，N 为表长），不难得出表长，而根据散列函数的选择要求，P 应该取不大于表长的最大素数，从而可以确定 P 的大小，也就构造出了散列函数。这里采用链地址法解决冲突，两种情况下的平均查找长度的计算过程与上一题完全相似。

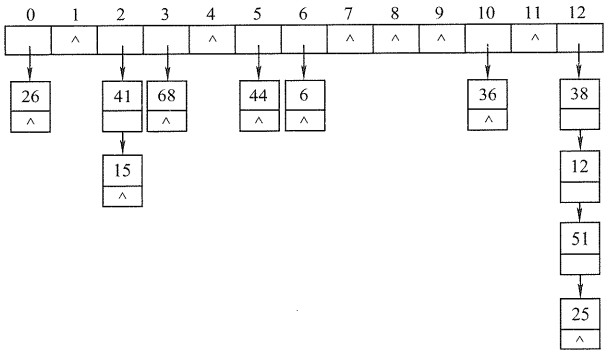
解答：

1）由 $\alpha = n/N$ 得 $N = n/\alpha$，由于 N 为整数，故应该向上取整，即 $N = \lceil n/\alpha \rceil = 15$，从而 $P = 13$。因此散列函数为 `H(key)=key%13`。

2）由 1）求出的散列函数，计算各关键字对应的散列地址如下表所示。

关键字	26	36	41	38	44	15	68	12	6	51	25
散列地址	0	10	2	12	5	2	3	12	6	12	12

由此构造的链地址法处理冲突的散列表为



由上图不难计算出

$$ASL_{成功} = (1\times7 + 2\times2 + 3\times1 + 4\times1)/11 = 18/11$$

$$ASL_{失败} = (1 + 0 + 2 + 1 + 0 + 1 + 1 + 0 + 0 + 0 + 1 + 0 + 4)/13 = 11/13$$

5．解答：

1）由装填因子 0.7 和数据总数 7，得一维数组大小为 7/0.7 = 10，数组下标为 0～9。所构造的散列函数值如下所示：

key	7	8	30	11	18	9	14
H(key)	0	3	6	5	5	6	0

采用线性探测再散列法处理冲突，所构造的散列表为

地址	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	

2）查找成功时，在等概率情况下，查找每个表中元素的概率是相等的。因此，根据表中元素的个数来计算平均查找长度，各关键字的比较次数如下所示：

key	7	8	30	11	18	9	14
次数	1	1	1	1	3	3	2

故 $ASL_{成功}$ = 查找次数/元素个数 =(1 + 2 + 1 + 1 + 1 + 3 + 3)/7 = 12/7。

在计算查找失败时的平均查找长度时，要特别注意防止思维定式，在查找失败的情况下既不是根据表中的元素个数，也不是根据表长来计算平均查找长度的。

查找失败时，在等概率情况下，经过散列函数计算后只可能映射到表中的 0～6 位置，且映射到 0～6 中任一位置的概率是相等的。因此，是根据散列函数（MOD 后面的数字）来计算平均查找长度的。在等概率情况下，查找失败的比较次数如下所示：

H(key)	0	1	2	3	4	5	6
次数	3	2	1	2	1	5	4

故 $ASL_{不成功}$ = 查找次数/散列后的地址个数 =(3 + 2 + 1 + 2 + 1 + 5 + 4)/7 = 18/7。

6．解答：

1）$H_0(2)=2$，$H_0(8)=8$，$H_0(31)=5$，$H_0(20)=7$，$H_0(19)=6$，没有冲突。$H_0(18)=5$，发生冲突，$H_1(18)=(H_0(18)+REV(18+1)\%11+1)\%13=(5+3+1)\%13=9$，没有冲突。$H_0(53)=1$，没有冲突。$H_0(27)=1$，发生冲突，$H_1(27)=(H_0(27)+REV(27+1)\%11+1)\%13=(1+5+1)\%13=7$，发生冲突，$H_2(27)=(H_1(27)+REV(27+1)\%11+1)\%13=0$，没有冲突。构造的

散列表如下：

散列地址	0	1	2	3	4	5	6	7	8	9	10	11	12
关键字	27	53	2			31	19	20	8	18			
比较次数	3	1	1			1	1	1	1	2			

2）由 1）中散列表的构造过程，各个关键字查找成功的比较次数如上表所示，故有

$$ASL_{成功}=(3+1+1+1+1+1+1+2)/8=11/8$$

归纳总结

本章的核心考查点是求平均查找长度（ASL），以度量各种查找算法的性能。查找算法本身依托于查找结构，查找结构又是由相同数据类型的记录或结点构成的，故最终落脚于数据结构类型的区别。

不管是何种查找算法，其平均查找长度的计算公式都是一样的。

查找成功的平均查找长度 $ASL_{成功}=\sum_{i=1}^{n}p_ic_i$。

查找失败的平均查找长度 $ASL_{不成功}=\sum_{j=0}^{n}q_jc_j$。

设一个查找集合中已有 n 个数据元素，每个元素的查找概率为 p_i，查找成功的数据比较次数为 c_i（$i=1,2,\cdots,n$）；不在此集合中的数据元素分布在由这 n 个元素的间隔构成的 $n+1$ 个子集合内，每个子集合元素的查找概率为 q_j，查找不成功的数据比较次数为 c_j（$j=0,1,\cdots,n$）。因此，对某一特定查找算法的查找成功的 $ASL_{成功}$和查找失败的 $ASL_{不成功}$，是综合考虑还是分开考虑呢？

若综合考虑，即 $\sum_{i=1}^{n}p_i+\sum_{j=0}^{n}q_j=1$，若所有元素查找概率相等，则有 $p_i=q_j=\frac{1}{2n+1}$；若分开考虑，即 $\sum_{i=1}^{n}p_i=1$，$\sum_{j=0}^{n}q_j=1$，若所有元素查找概率相等，则有 $p_i=\frac{1}{n}$，$q_j=\frac{1}{n+1}$。

虽然综合考虑更为理想，但在实际应用中多数是分开考虑的，因为对于查找不成功的情况，很多场合下没有明确给出，往往会被忽略掉。不过读者仍要注意的是，这两种考虑的计算结果是不同的，考试中一定要仔细阅读题目的要求，以免失误。

思维拓展

本章介绍了几种基本的查找算法，在实际中又会碰到怎样的查找问题呢？

题目：数组中有一个数字出现的次数超过了数组长度的一半，请找出这个数字。读者也许会想到先进行排序，位于位置$(n+1)/2$ 的数即为要找的数，这样最小时间复杂度就为 $O(n\log_2 n)$；若进行散列查找，数字的范围又未知，那么应如何将时间复杂度控制在 $O(n)$内呢？

（**提示**：出现的次数超过数组长度的一半，表明这个数字出现的次数比其他数字出现的次数的总和还多。所以我们可以考虑每次删除两个不同的数，则在剩下的数中，待找数字出现的次数仍然超过总数的一半。通过不断重复这个过程，不断排除其他的数字，最终剩下的都为同一个数字，即为要找的数字。）